Data mining – Notes

Francesco Passino

Contents

1 Lesson 01 — 22/09/2026 **4**
1.1 Representation and types . . . 4
1.2 Statistical description of data . . . 5
1.2.1 Central tendency . . . 5
1.2.2 Measuring the dispersion of data . . . 5
1.2.3 Graphic displays . . . 7
1.3 Data visualization . . . 7
1.3.1 Pixel-oriented techniques . . . 8
1.3.2 Geometric projection techniques . . . 8
1.3.3 Icon-based techniques . . . 8
1.3.4 Hierarchical techniques . . . 8
1.3.5 Visualizing complex relations . . . 9
1.4 Similarity and dissimilarity . . . 9
1.4.1 Data matrix and dissimilarity matrix . . . 9
1.4.2 Proximity for nominal attributes . . . 10
1.4.3 Proximity for binary attributes . . . 10
1.4.4 Proximity for numeric attributes . . . 11
1.4.5 Proximity for ordinal attributes . . . 13
1.4.6 Dissimilarity for attributes of mixed types . . . 13
1.4.7 Cosine similarity . . . 14
1.4.8 Choosing a proximity measure . . . 15
1.5 Practical checklist . . . 15

2 Data pre-processing – lesson 29/09/2026 **16**
2.1 Data quality . . . 17
2.2 Data cleaning . . . 17
2.2.1 Missing values . . . 17
2.2.2 Noise and smoothing . . . 18
2.2.3 Data cleaning as a process . . . 21
2.3 Data integration . . . 22
2.4 Redundancy . . . 22

3 Lesson 03 — 06/10/2026 **25**
3.1 Correlation Analysis with numeric data . . . 25
3.1.1 Covariance . . . 25
3.2 Data reduction . . . 26
3.3 Principal Component Analysis (PCA) . . . 27
3.4 Subset selection . . . 29
3.4.1 An example of heuristic: NMIFS . . . 30
3.5 Attribute creation / feature generation . . . 31
3.6 Numerosity reduction . . . 31

3.6.1 Parametric methods . . . 31
3.6.2 Non-parametric methods . . . 32
3.6.3 Aggregation and compression . . . 34
3.7 Transformation . . . 34
3.7.1 Discretization . . . 35

4 Lesson 04 — 13/10/2026 **38**
4.1 Classification . . . 38
4.1.1 Editing methods . . . 40
4.2 Prediction problems . . . 41
4.3 Case-Based Reasoning (CBR) . . . 41

5 Lesson 05 — 15/10/2026 **42**
5.1 Eager learners . . . 42
5.2 Linear support vector machine . . . 42
5.3 Nonlinear SVM and the kernel idea . . . 43
5.4 Decision tree . . . 44
5.4.1 Algorithm for decision tree induction . . . 44

6 Lesson 06 — 20/10/2026 **45**
6.1 Some notation . . . 45
6.1.1 Split-point search . . . 46
6.2 Decision tree induction: top-down greedy learning . . . 46
6.2.1 Gain ratio . . . 46
6.3 Overfitting . . . 47
6.4 Pruning . . . 47
6.4.1 Reduced error pruning . . . 48
6.4.2 Interpretability . . . 48
6.5 Bayesian classifier . . . 48
6.6 Bayesian belief networks . . . 49

7 Lesson 07 — 22/10/2026 **49**
7.1 Rule-based Classification . . . 49
7.1.1 Rules from decision trees . . . 50
7.1.2 Sequential covering . . . 50
7.2 Model evaluation and selection . . . 51
7.2.1 Area under curve . . . 52

8 Lesson 08 — 03/11/2026 **53**
8.1 Cost of a classifier . . . 53
8.2 Estimating Classifier Accuracy . . . 54
8.2.1 Null hypothesis and paired comparison . . . 55
8.3 Paired t-test . . . 55

9 Lesson 09 — 05/11/2026 **55**
9.1 Wilcoxon signed-rank test . . . 55
9.1.1 Statistic and p-value . . . 56
9.2 Neural Networks . . . 57
9.2.1 Multilayer feedforward networks . . . 57
9.2.2 Backpropagation . . . 58
9.3 Ensemble Methods . . . 58
9.3.1 Bagging . . . 58

10 Lesson 10 — 10/11/2026 **59**
10.1 Boosting . . . 59
10.2 Random forest . . . 60
10.3 Imbalanced datasets . . . 61
10.3.1 How to choose neighbours . . . 61
10.3.2 Generating one synthetic sample . . . 61

11 Lesson 11 — 12/11/2026 **62**
11.1 Clustering . . . 62
11.2 Partitioning algorithms . . . 63
11.2.1 k-means . . . 64
11.2.2 k-medoids . . . 65

1 Lesson 01 — 22/09/2026

We start by getting to know the data, how are they represented, which different type we have and some basic statistical descriptions and useful data visualization. We will then introduce the concept of similarity and dissimilarity using the similarity matrix.

1.1 Representation and types

Data sets are made up of `data objects`, a data object represents an entity and are typically described by `attributes`. The `type` of an attribute is determined by the set of possible values the attribute can have.

- `Nominal attributes`: the value of nominal attribute are symbols or names of things, each value represents some kind of category or state and there isn't any meaningful ordering.
- `Binary attributes`: it's a nominal attribute with only two category or state (0 and 1). Can be *simmetric* if both states equally matter or *asymmetric* if one state is more informative or rare then the other.
- `Ordinal attributes`: the different values have a meaningful order or *ranking* but the gaps between successive values are not quantified.
- `Numeric attributes`: a numeric attribute is *quantitive* and we can measure its quantity and represents it with integer or real values. Numeric attributes can be `Intervale-scaled`, if they are measured on an equal-size unites scale without a true 0, or `Ratio-scaled`, if there is an inherent zero-point and we can measure a value as being a multiple of another value.

In data mining also we categorize attributes as being either *discrete* or *continuos*. A `discrete attribute` as a finite or countably infinite set of values. An attribute is countably infinite if the set of possible values is infinite but the values can be put in a one-to-one correspondence with natural numbers. If an attribute is not discrete, it is `continuos`, and it is typically represented as a floating-point variable.

Data objects are also called *samples*, *examples*, *instances*, *data points* or *tuples*, while attributes are also called *dimensions*, *features* or *variables*. When the data are stored in a database, the rows correspond to the data objects and the columns to the attributes.

The type of an attribute tells us which operations are meaningful on its values:

- nominal (and binary): only equality tests and counting, so the only measure of central tendency we can use is the mode;
- ordinal: we can also sort the values, so the median is defined too, but the mean is not;
- interval-scaled: differences are meaningful and we can compute the mean, but ratios make no sense: 20°C is 10°C warmer than 10°C, but it is not "twice as hot";
- ratio-scaled: ratios are meaningful as well (length, counts, money, temperature in Kelvin).

Remark. Coding the categories of a nominal attribute as $0, 1, 2, \ldots$ does not make it numeric: the codes carry no ordering and arithmetic on them is not meaningful (it makes no sense to subtract one customer ID from another). In the same way, an attribute stored as an "integer" in a database is not necessarily numeric in the statistical sense, as happens for ZIP codes.

Focusing on attribute types is not a limitation. Complex raw data such as images, text, audio signals and sensor streams are first encoded as numbers (pixel matrices, tokens and counts, embeddings), and each resulting feature is still nominal, ordinal, interval or ratio: its scale tells us which mathematical operations remain valid once the data have been encoded.

1.2 Statistical description of data

A basic statistical description is essential to have an overall picture of the data and can be used to identify properties, valuable data, noise and outliers.

1.2.1 Central tendency

We will se various ways to measure the central tendency of data that include the mean, median and mode. The most common measure of the "center" of a set of data is the `arithmetic mean`. Let $x_1, x_2, \ldots, x_N$ be a set of N values, the arithmetic mean is

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_N}{N} = \frac{\sum_{i=1}^{N} x_i}{N}.$$

Sometimes, each value x_i in a set may be associated with a weight w_i that reflects the significance,importance or frequency attached to their respective values. In this case we can compute the `weighted mean` as

$$\bar{x} = \frac{w_1 x_1 + w_2 x_2 + \cdots + w_N x_N}{w_1 + w_2 + \cdots + w_N} = \frac{\sum_{i=1}^{N} w_i x_i}{\sum_{i=1}^{N} w_i}.$$

Although the mean is the single most useful quantity for describing a data set, it's sensitive to extreme values (e.g. outliers). We can instead use the `trimmed mean`, which is the arithmetic mean obtained after chopping off extreme high and low values.

For asymmetric data, a better measure of the center of data is the `median`, which is the middle value in a set of ordered values. It generally applies to numeric data but we can extend the concept to ordinal data.

Mode is another measure of central tendency. The `mode` for a set of data is the value that occurs most frequently in the set. It works for nominal data where we can identify more then one mode, the distribution can be unimodal, bimodal or multimodal.

When the number N of observations is even the median is not unique: by convention we take the average of the two middle values. Unlike the mean, the median is robust to extreme values, so we prefer it when the distribution is skewed or heavy-tailed; the mode is the right choice when frequency, and not arithmetic distance, is what matters.

Comparing mean, median and mode reveals the `skewness` of a unimodal distribution:

- *symmetric* data (balanced tails): mean $\approx$ median $\approx$ mode;
- *positively skewed* data (longer right tail): mode $<$ median $<$ mean;
- *negatively skewed* data (longer left tail): mean $<$ median $<$ mode.

1.2.2 Measuring the dispersion of data

We can measure the dispersion or spread of data through p`percentiles` and `quartiles`. The kth percentile is the value such that approximately $k\%$ of observations lie at or below it. Quantiles are points taken at regular intervals of data distribution, dividing it into equal-size consecutive sets. For example the 4-quantiles(`quartiles`) are the three data points that split the data distribution into four equal parts; each part represents one-fourth of the data distribution. We ca use quantile plot to visualize information on the distribution: sort the observations from smallest to largest; plot each value against a cumulative fraction f; read quartiles at $f = 0.25$, $f = 0,50$ and $f = 0.75$

More precisely, in a quantile plot the i-th sorted observation x_i is plotted against

$$f_i = \frac{i - 0.5}{N},$$

which means that approximately $f_i \cdot 100\%$ of the data lie below x_i. The horizontal axis is therefore not time but the fraction of data at or below a value. Since the plot shows all the observations, jumps, clusters, gaps and outliers remain visible.

The three quartiles are denoted by Q_1 (25th percentile), Q_2 (50th percentile, i.e. the median) and Q_3 (75th percentile). The distance between the first and the third quartile is the `interquartile range`

$$IQR = Q_3 - Q_1,$$

which summarizes the central 50% of the data and is robust to extreme values.

To visualize distribution it's also commonly used teh `box plot`:

- The ends of the box are the quartiles (Q1 and Q3)
- The median is marked by a line in the box
- Two lines (called whiskers) outside the box extend to the smallest and largest observations.

Typically whiskers are extended within 1,5xIQR(Inter Quartile Range) and points beyond the whiskers are plotted individually and they represent the outliers.

In detail, the usual rule of thumb treats as suspected outliers the values lying more than $1.5 \cdot IQR$ below Q_1 or above Q_3, and the whiskers stop at the most extreme observations inside these limits. The box plot is thus a picture of the `five-number summary` of a distribution, which is made of *Minimum*, Q_1, *Median*, Q_3, *Maximum*.

Drawing the box plots of several data sets side by side lets us compare the distributions at a glance, without showing every data point: the median line gives the typical value, the height of the box the spread of the middle 50% of the values, the whiskers the overall spread and the isolated points the possible outliers.

To quantify spread around the mean we can use `variance` adn `standard deviation`. The variance of N observations $x_1, x_2, x, \ldots, x_N$ is:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2$$

where μ is the mean, squaring prevents positive and negative deviations from cancelling out. The standard deviations, σ, is the square root of the variance. Because both depend on the mean, variance and standard variation can be strongly influenced by outliers.

For a normal distribution the standard deviation has a direct reading, the `68-95-99.7 rule`: about 68% of the values lie in the interval $\mu \pm \sigma$, about 95% in $\mu \pm 2\sigma$ and about 99.7% in $\mu \pm 3\sigma$. The rule is about intervals that are symmetric around the mean.

Example 1.1 (Salary data)**.**
Consider the following $N = 12$ salaries (in thousands of dollars), already sorted:

$$30,\ 36,\ 47,\ 50,\ 52,\ 52,\ 56,\ 60,\ 63,\ 70,\ 70,\ 110.$$

- Mean: $\bar{x} = 696/12 = 58$.
- Median: N is even, so we average the 6th and the 7th value, $(52 + 56)/2 = 54$.
- Mode: the data are bimodal, with modes 52 and 70.
- Quartiles: $Q_1 = (47 + 50)/2 = 48.5$ and $Q_3 = (63 + 70)/2 = 66.5$, hence $IQR = 66.5 - 48.5 = 18$.
- Variance: $\sigma^2 = \frac{1}{12}\left(30^2 + 36^2 + \cdots + 110^2\right) - 58^2 \approx 379.17$, hence $\sigma \approx 19.47$.

The mean is larger than the median because it is pulled up by the extreme value 110. With the $1.5 \cdot IQR$ rule every value above $66.5 + 27 = 93.5$ is a suspected outlier, so in a box plot 110 would be plotted individually.

1.2.3 Graphic displays

A distribution is described by its *center* (where are the typical values?), its *spread* (how variable are they?), its *shape* (symmetric, skewed, multimodal?) and its *tails* (are there outliers or rare events?). A single number is never the whole story, so we combine the statistics above with a visual inspection of the data. Quantile plots and box plots were introduced above; here we see the other common displays.

A `histogram` summarizes the distribution of a numeric attribute: the range of values is partitioned into disjoint consecutive subranges, called *buckets* or *bins* and typically of equal width, and for each bin we draw a bar whose height is the number of observations falling in it. In a histogram we look for the number of modes, skewness, gaps, heavy tails and potential outliers, keeping in mind that its appearance is sensitive to the bin width. A histogram is different from a *bar chart*: the bars of a bar chart represent categories and can be reordered, while the bars of a histogram represent adjacent numeric intervals. Moreover it is the area of each bar that represents the value and not just its height, which matters when the bins have different widths.

Histograms often tell more than box plots. A box plot summarizes position and spread, but it can hide shape, modes and gaps: an approximately uniform sample and a clustered, multimodal one can share the same five-number summary (e.g. *Minimum* $= 0$, $Q_1 = 2$, *Median* $= 4$, $Q_3 = 6$, *Maximum* $= 8$) and so the same box plot, while their histograms are clearly different.

A `quantile-quantile plot` (or `q-q plot`) graphs the quantiles of one univariate distribution against the corresponding quantiles of another, so we can see whether there is a shift going from one distribution to the other. Given two sorted samples $x_1, \dots, x_N$ and $y_1, \dots, y_M$, if $M = N$ we simply plot y_i against x_i, since both are the $(i - 0.5)/N$ quantiles of their data sets; if $M < N$ we plot only M points, pairing y_i with the $(i - 0.5)/M$ quantile of the x data, which usually has to be computed by interpolation. The diagonal line marks the case where the two distributions have equal values at each quantile: if, for example, most of the points lie above the diagonal, the distribution on the vertical axis tends to have higher values at the same percentiles.

A `scatter plot` gives a first look at bivariate data: each pair of values of two numeric attributes is treated as a pair of coordinates and plotted as a point in the plane. It lets us see clusters of points and outliers and explore whether the two attributes are correlated. If the cloud of points slopes from lower left to upper right the association is *positive*, if it slopes from upper left to lower right it is *negative*, otherwise there is no clear linear association.

Remark. Correlation describes association, not causation. Moreover a zero linear correlation can still hide a nonlinear structure in the data.

1.3 Data visualization

Visualization is a reasoning tool. We use it to build an immediate qualitative *overview* of a large data set, to search for *patterns* (trends, clusters, structure and anomalies), for *diagnostics* (spotting data quality issues and choosing the parameters of later analyses) and for *communication*, since it makes the inputs, the outputs and the evidence of a model easier to inspect.

Different families of techniques emphasize different aspects of high-dimensional data:

- `Pixel-oriented`: one tiny mark per value.
- `Geometric projection`: the dimensions are projected or arranged in a geometric space.
- `Icon-based`: the attributes are mapped to the visual properties of a glyph.

- `Hierarchical`: the space is partitioned according to a hierarchy of the dimensions.
- `Complex relations`: the focus is on text, networks or linked structures.

1.3.1 Pixel-oriented techniques

They are useful when there are too many values to draw each of them with a conventional mark. For a data set with m dimensions we create m windows on the screen, one for each dimension; the m values of a record are mapped to m pixels placed at the same position in each window, and the color of a pixel reflects the corresponding value. If, for example, the customers are sorted by income, the same pixel position in the windows of income, credit limit, transaction volume and age refers to the same customer. Because the ordering is fixed, dark or light areas that recur in several windows indicate a relationship between the attributes.

1.3.2 Geometric projection techniques

A `scatter-plot matrix` extends the scatter plot to several numeric attributes: it is a grid containing the scatter plot of every pair of attributes. It is a compact overview that we use to scan many pairs at once, to detect strong linear or nonlinear relationships, to identify clusters and outliers and to decide which individual scatter plots deserve a closer look.

With `parallel coordinates` we draw one vertical axis per attribute, each scaled to the $[minimum, maximum]$ range of its attribute, and every observation becomes a polyline that crosses each axis at the corresponding value. We look for bundles of similar lines, crossings, outlying trajectories and attributes that separate groups: parallel, bundled lines indicate similar profiles, while crossings between two axes suggest that the two attributes rank the objects differently. The strength of this technique is that it preserves the full high-dimensional record; its weakness is that it quickly becomes cluttered as the number of observations grows.

1.3.3 Icon-based techniques

`Chernoff faces` map the attributes to the features of a face, such as the size of the eyes, the curvature of the mouth and the angle of the eyebrows. Humans are very sensitive to facial patterns, but the interpretation is subjective.

`Stick figures` map the attributes to the lengths and the angles of the limbs of a small figure. Typically two attributes define the position of the figure on the plane and the remaining ones are encoded by the body and the limbs: we read the single glyph for the individual values and the overall texture, created by the repeated glyphs, for patterns, clusters and unusual records in very dense multivariate data.

In general every visual encoding introduces perceptual assumptions, and some variables are easier to compare than others.

1.3.4 Hierarchical techniques

`Dimensional stacking` uses nested two-dimensional partitions to show more than two attributes at once: we partition the first two attributes in a grid of cells and, inside each cell, we recursively partition the next two attributes. It works best for ordinal attributes and for binned numeric attributes with low or medium cardinality; the result depends strongly on the order of the dimensions and on the bin boundaries, and readability drops when the dimensions are many.

A `tree-map` uses area to show the part-to-whole structure of hierarchical data: each rectangle is an item, its area encodes the magnitude, the nesting of the rectangles encodes the hierarchy and the color can encode a second variable. For example, if a node A of value 200 has two children $B = 80$ and $C = 120$, the rectangle of A is split in two parts that cover 40% and 60% of its area, and each of them is split again among its own children. Tree-maps are well suited to file systems, portfolios, inventories and hierarchical categories.

1.3.5 Visualizing complex relations

A `tag cloud` is a visualization of text in which each word (or tag) is displayed with a size proportional to a chosen weight, usually its frequency. Typical sources are user-generated tags, keywords extracted from documents and open answers or comments. The largest tags reveal the dominant topics, but similar sizes allow only an approximate comparison and the position of a word has no precise meaning. A tag cloud does not show the relationships between words, their temporal order or the context in which a term appears: we use it for an overview and not for measurement.

1.4 Similarity and dissimilarity

In data mining applications, such as clustering, outlier analysis, and nearest-neighbor classification, we need ways to assess how alike or unalike objects are in comparison to one another. Similarity and dissimilarity are both measure of *proximity*.

A `similarity` measure is a number that tells how alike two objects are: the larger the value, the more alike the objects. It is often normalized to $[0, 1]$, where 0 means that the objects are completely unalike and 1 that they are identical. A `dissimilarity` (or `distance`) measure works the opposite way: the smaller the value, the more alike the objects. Its minimum is usually 0, reached when the objects are the same, while the upper bound depends on the measure.

1.4.1 Data matrix and dissimilarity matrix

So far the objects were described by a single attribute. From now on we have n objects described by p attributes: object i is $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})$, where x_{if} is its value for the f-th attribute. We work with two data structures.

The `data matrix` (object-by-attribute structure) stores the n objects as an $n \times p$ matrix, one row per object:

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \vdots & & \vdots & & \vdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \vdots & & \vdots & & \vdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}.$$

The `dissimilarity matrix` (object-by-object structure) stores the proximities of all the pairs of objects as an $n \times n$ table:

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & \ddots & \\ d(n,1) & d(n,2) & \cdots & \cdots & 0 \end{bmatrix}$$

where $d(i, j)$ is the dissimilarity between objects i and j: a nonnegative number, close to 0 when the two objects are very similar and larger the more they differ. Since $d(i, i) = 0$ and $d(i, j) = d(j, i)$, the matrix is symmetric and we write only its lower triangle.

The data matrix contains two kinds of entities (objects on the rows, attributes on the columns) and is called a *two-mode* matrix, while the dissimilarity matrix contains only one kind and is called a *one-mode* matrix. Many clustering and nearest-neighbor algorithms operate on a dissimilarity matrix, so a data matrix must be transformed into a dissimilarity matrix before applying them. A similarity can often be obtained from a dissimilarity, for example as

$$sim(i, j) = 1 - d(i, j).$$

How $d(i, j)$ is computed depends on the type of the attributes.

1.4.2 Proximity for nominal attributes

For nominal attributes we count the matches, without inventing an arithmetic distance between the codes of the categories. The dissimilarity between two objects i and j is the ratio of mismatches

$$d(i,j) = \frac{p-m}{p},$$

where m is the number of *matches*, i.e. the number of attributes for which i and j are in the same state, and p is the total number of attributes. The corresponding similarity is

$$sim(i,j) = 1 - d(i,j) = \frac{m}{p}.$$

Example 1.2.
Two objects are described by eye color, hair color and region. Object i is (green, blond, north) and object j is (green, black, north): there are $m = 2$ matches out of $p = 3$ attributes, so

$$d(i,j) = \frac{3-2}{3} \approx 0.33, \qquad sim(i,j) = \frac{2}{3} \approx 0.67.$$

An alternative is `one-hot encoding`: for a nominal attribute with M states we create M binary attributes, one for each state, and for a given object exactly one of them is set to 1 (the one of its state) while the others are set to 0. The encoded vectors are then compared with the measures for binary attributes of the next section. So a category such as "eye = blue" becomes the Boolean statement "is the eye blue?". The caveat is that, when the binary attributes are many, most of them are 0 for both objects and these common zeros can inflate the similarity: we should either compare the matches at the level of the original attributes or choose a suitable binary coefficient, one that ignores the joint absences.

1.4.3 Proximity for binary attributes

Treating binary attributes as if they were numeric can be misleading, and symmetric and asymmetric attributes must be treated differently. Given two objects i and j described by p binary attributes, we count the attributes in the four possible situations and we collect the counts in a contingency table:

	$j=1$	$j=0$
$i=1$	q	r
$i=0$	s	t

so q is the number of attributes equal to 1 for both objects, r the number of those equal to 1 for i and 0 for j, s the number of those equal to 0 for i and 1 for j, and t the number of those equal to 0 for both, with $p = q + r + s + t$. Hence $q + t$ are the matches and $r + s$ the mismatches.

For *symmetric* binary attributes both states matter equally and we use the `symmetric binary dissimilarity` (simple matching distance)

$$d_{SMC}(i,j) = \frac{r+s}{q+r+s+t},$$

whose complement $(q+t)/(q+r+s+t)$ is the *simple matching coefficient* (SMC).

For *asymmetric* binary attributes the presence (1) matters more than the joint absence: a match of two 1s is more significant than a match of two 0s. The number t of negative matches is considered unimportant and is ignored, which gives the `asymmetric binary dissimilarity`

$$d_J(i,j) = \frac{r+s}{q+r+s}.$$

The corresponding similarity is the `Jaccard coefficient`

$$J(i,j) = \frac{q}{q+r+s} = 1 - d_J(i,j).$$

Example 1.3.
Two patients share 3 positive symptoms ($q = 3$), differ on 2 ($r + s = 2$) and share 20 absent symptoms ($t = 20$). Then

$$\text{SMC} = \frac{q+t}{q+r+s+t} = \frac{23}{25} = 0.92, \qquad J = \frac{q}{q+r+s} = \frac{3}{5} = 0.60.$$

The SMC is dominated by the joint absences, while the Jaccard coefficient focuses on the informative presences: for sparse asymmetric data "both absent" carries little evidence of similarity.

Example 1.4.
We can now see the caveat of one-hot encoding. Object i has blue eyes and brown hair, object j has green eyes and brown hair: at the level of the original attributes there is 1 match out of 2 attributes, so $sim(i,j) = 0.5$. If we encode eye color with three binary attributes (black, green, blue) and hair color with four (auburn, black, blond, brown), we get

$$i = (0,0,1,0,0,0,1), \qquad j = (0,1,0,0,0,0,1),$$

that is $q = 1$, $r = 1$, $s = 1$ and $t = 4$. The simple matching coefficient is $(q+t)/7 = 5/7 \approx 0.71$, inflated by the four common zeros, while the Jaccard coefficient is $1/3 \approx 0.33$.

1.4.4 Proximity for numeric attributes

For objects described by numeric attributes we use a family of distances.

Definition 1.5 (Minkowski distance)**.**
Let $i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ and $j = (x_{j1}, x_{j2}, \ldots, x_{jp})$ be two objects described by p numeric attributes. The Minkowski distance between i and j is

$$d(i,j) = \left(\sum_{f=1}^{p} |x_{if} - x_{jf}|^h \right)^{1/h},$$

where h is a real number such that $h \geq 1$. It is also called the L_h norm.

The most used special cases are:

- $h = 1$, the `Manhattan` (or city block) distance, the sum of the absolute differences of the coordinates:
$$d(i,j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \cdots + |x_{ip} - x_{jp}|.$$
It is useful when movements or costs accumulate dimension by dimension. On binary vectors it becomes the *Hamming distance*, which counts the differing bits.

- $h = 2$, the `Euclidean` distance, the straight-line distance in the geometric space:
$$d(i,j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \cdots + (x_{ip} - x_{jp})^2}.$$
It is the most common choice, but it is highly sensitive to the scale of the attributes and it becomes less and less discriminative in high dimensions.

- $h \to \infty$, the `supremum` (or Chebyshev, L_∞) distance, the maximum absolute difference across the attributes:

$$d(i,j) = \lim_{h\to\infty} \left(\sum_{f=1}^{p} |x_{if} - x_{jf}|^h \right)^{1/h} = \max_f |x_{if} - x_{jf}|.$$

  It is useful when the worst deviation on a single dimension dominates the notion of dissimilarity.

These distances satisfy the following properties:

- *Non-negativity*: $d(i,j) \geq 0$.
- *Identity*: $d(i,j) = 0$ if and only if $i = j$; in particular the distance of an object from itself is 0.
- *Symmetry*: $d(i,j) = d(j,i)$.
- *Triangle inequality*: $d(i,j) \leq d(i,k) + d(k,j)$, i.e. going directly from i to j is never longer than making a detour through another object k.

A measure that satisfies these properties is called a `metric`.

Example 1.6.
Consider four points described by two attributes:

point	attribute 1	attribute 2
x_1	1	2
x_2	3	5
x_3	2	0
x_4	4	5

For x_1 and x_2 the differences on the two attributes are 2 and 3, so the Manhattan distance is $2 + 3 = 5$, the Euclidean distance is $\sqrt{2^2 + 3^2} \approx 3.61$ and the supremum distance is $\max\{2, 3\} = 3$. Repeating the computation for all the pairs we obtain the three dissimilarity matrices (in this order: Manhattan, Euclidean, supremum):

$$\begin{bmatrix} 0 & & & \\ 5 & 0 & & \\ 3 & 6 & 0 & \\ 6 & 1 & 7 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & & & \\ 3.61 & 0 & & \\ 2.24 & 5.10 & 0 & \\ 4.24 & 1 & 5.39 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & & & \\ 3 & 0 & & \\ 2 & 5 & 0 & \\ 3 & 1 & 5 & 0 \end{bmatrix}$$

Attributes measured on large scales can dominate a distance: if income ranges in 0–100 000 and age in 0–100, the Euclidean distance will mostly measure income. For this reason we usually `standardize` the data before computing distances, replacing each value x of an attribute with its *z-score*

$$z = \frac{x - \mu}{\sigma},$$

where μ and σ are the mean and the standard deviation of that attribute. The z-score is a unitless variable, so standardization puts the attributes on comparable scales. When the outliers are strong, robust alternatives use the median and the median absolute deviation in place of μ and σ.

Remark. The scaling parameters must be fitted on the training set only and then applied to the validation and test data.

1.4.5 Proximity for ordinal attributes

The values of an ordinal attribute have a meaningful ranking, but the gap between adjacent categories is not defined by the data. We treat ordinal attributes almost like numeric ones. Let f be an ordinal attribute with M_f ordered states:

1. we replace the value x_{if} of each object with its rank $r_{if} \in \{1, \dots, M_f\}$;
2. since different ordinal attributes can have a different number of states, we normalize the ranks to $[0, 1]$, so that every attribute has the same weight, replacing r_{if} with
$$z_{if} = \frac{r_{if} - 1}{M_f - 1};$$
3. we compute the dissimilarity with any of the distances for numeric attributes, using z_{if} as the value of attribute f for object i.

After the normalization the ordinal attributes can also be combined with numeric attributes in the same distance.

Example 1.7.
For the attribute size with states small $<$ medium $<$ large we have $M_f = 3$; the ranks are $1, 2, 3$ and the normalized values are 0, 0.5 and 1. Suppose now that four objects have, for an ordinal attribute with states fair $<$ good $<$ excellent, the values excellent, fair, good, excellent. Their ranks are $3, 1, 2, 3$, the normalized values are $1, 0, 0.5, 1$ and, with the Euclidean distance, the dissimilarity matrix is

$$\begin{bmatrix} 0 & & & \\ 1.0 & 0 & & \\ 0.5 & 0.5 & 0 & \\ 0 & 1.0 & 0.5 & 0 \end{bmatrix}$$

Objects 1 and 4 are identical, while object 2 is at the opposite end of the range with respect to both.

Remark. This transformation treats consecutive ranks as equally spaced. It is a modeling assumption and not an information contained in the original categories.

1.4.6 Dissimilarity for attributes of mixed types

Real data sets often contain numeric, nominal, binary and ordinal attributes together. Each attribute should be compared with a distance suitable for its type, and the single contributions are then combined in one dissimilarity matrix, bringing all of them onto the common scale $[0, 1]$. If the data set contains p attributes of mixed types, the dissimilarity between objects i and j is

$$d(i, j) = \frac{\sum_{f=1}^{p} \delta_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^{p} \delta_{ij}^{(f)}},$$

where the indicator $\delta_{ij}^{(f)}$ is 0 if x_{if} or x_{jf} is missing, or if $x_{if} = x_{jf} = 0$ and f is an asymmetric binary attribute, and it is 1 otherwise. The contribution $d_{ij}^{(f)}$ of attribute f depends on its type:

- f numeric: $d_{ij}^{(f)} = \dfrac{|x_{if} - x_{jf}|}{\max_f - \min_f}$, where $\max_f$ and $\min_f$ are the maximum and the minimum value of attribute f;
- f nominal or binary: $d_{ij}^{(f)} = 0$ if $x_{if} = x_{jf}$, otherwise $d_{ij}^{(f)} = 1$;

- f ordinal: we compute the ranks r_{if} and $z_{if} = \dfrac{r_{if} - 1}{M_f - 1}$, and we treat z_{if} as numeric.

More in general the final dissimilarity can be a weighted average of the attribute-level dissimilarities,

$$d(i, j) = \frac{\sum_f w_f \, d_{ij}^{(f)}}{\sum_f w_f},$$

where the weights w_f encode the importance, the reliability or the availability of each attribute; the formula above is the case in which the weights are the indicators $\delta_{ij}^{(f)}$. For example, the similarity between two patients may combine a distance on the age, the matching of the diagnosis, the Jaccard similarity of the symptoms and the rank of the severity of the disease.

Example 1.8.
Consider four objects described by three attributes of different types:

object	test-1 (nominal)	test-2 (ordinal)	test-3 (numeric)
1	code A	excellent	45
2	code B	fair	22
3	code C	good	64
4	code A	excellent	28

No value is missing and there are no asymmetric binary attributes, so $\delta_{ij}^{(f)} = 1$ everywhere. Let us compute $d(3, 1)$. For test-1 the two codes differ, so $d_{31}^{(1)} = 1$. For test-2 the normalized ranks are 0.5 and 1, so $d_{31}^{(2)} = 0.5$. For test-3 we have $\max_f - \min_f = 64 - 22 = 42$, so $d_{31}^{(3)} = |64 - 45|/42 \approx 0.45$. Then

$$d(3, 1) = \frac{1 + 0.50 + 0.45}{3} = 0.65.$$

Repeating the computation for all the pairs we obtain

$$\begin{bmatrix} 0 & & & \\ 0.85 & 0 & & \\ 0.65 & 0.83 & 0 & \\ 0.13 & 0.71 & 0.79 & 0 \end{bmatrix}$$

Objects 1 and 4 are the most similar, as we could guess from their values of test-1 and test-2, while objects 1 and 2 are the least similar.

1.4.7 Cosine similarity

A document can be represented by a *term-frequency vector*, with one attribute for each word that records how many times the word occurs in the document. These vectors are very long and *sparse*, i.e. they have many 0 values, and the distances seen so far do not work well on them: two documents can have many 0 values in common, but this does not make them similar. We need a measure that focuses on the words that the two documents do have in common and on their frequency, ignoring the zero-matches.

Definition 1.9 (Cosine similarity)**.**

Let d_1 and d_2 be two vectors (e.g. two term-frequency vectors). Their cosine similarity is

$$\cos(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \, \|d_2\|},$$

where $d_1 \cdot d_2$ is the dot product of the two vectors and $\|d\|$ is the Euclidean norm (the length) of the vector $d = (x_1, \ldots, x_p)$, that is $\|d\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_p^2}$.

The measure is the cosine of the angle between the two vectors, so it focuses on their direction and not on their magnitude: a value close to 1 means that the vectors point in the same direction, a value close to 0 that they are orthogonal, i.e. unrelated. For this reason it is useful when the length of the documents varies, and it is also used for term-frequency or TF-IDF vectors, embeddings and recommendation vectors. The cosine similarity does not satisfy all the properties of a metric, so it is a *nonmetric* measure.

Example 1.10.
Consider the term-frequency vectors of two documents over the words team, coach, hockey, baseball, soccer, penalty, score, win, loss, season:

$$d_1 = (5, 0, 3, 0, 2, 0, 0, 2, 0, 0), \qquad d_2 = (3, 0, 2, 0, 1, 1, 0, 1, 0, 1).$$

We have

$$\begin{aligned} d_1 \cdot d_2 &= 5 \cdot 3 + 3 \cdot 2 + 2 \cdot 1 + 2 \cdot 1 = 25, \\ \|d_1\| &= \sqrt{5^2 + 3^2 + 2^2 + 2^2} = \sqrt{42} \approx 6.481, \\ \|d_2\| &= \sqrt{3^2 + 2^2 + 1^2 + 1^2 + 1^2 + 1^2} = \sqrt{17} \approx 4.123, \end{aligned}$$

and therefore

$$\cos(d_1, d_2) = \frac{25}{6.481 \cdot 4.123} \approx 0.94.$$

The two documents use many of the same terms in similar proportions, so they are quite similar. The Euclidean distance, instead, can be strongly affected by the length of the documents, while the cosine similarity compares only the orientation of the vectors.

1.4.8 Choosing a proximity measure

The proximity measure must match the type of the data:

Data type	Typical choice	Caution
Numeric continuous	Euclidean, Manhattan, Mahalanobis	scale the variables first
Sparse text vectors	cosine, Jaccard	direction matters more than magnitude
Asymmetric binary	Jaccard	ignore the joint absences
Nominal categories	matching, one-hot + suitable metric	avoid fake orderings
Ordinal variables	rank-based distance	gaps between ranks are assumptions
Mixed attributes	weighted mixed-type distance	choose the weights carefully

1.5 Practical checklist

Before modeling we should be able to answer the following questions about our data:

1. *What is an object?* What does one row, or one instance, represent?
2. *What is each attribute?* What are its measurement scale and its semantic meaning?

3. *What is the distribution?* Central tendency, spread, skewness, missing values and outliers.
4. *How is similarity defined?* Which distance or similarity measure matches the task?
5. *What can go wrong?* Leakage, biased samples, duplicated objects, wrong units, hidden groups.
6. *What should be visualized?* Raw data, transformed data and model errors.

The most common pitfalls in data exploration are:

- treating identifiers as numeric features;
- using the Euclidean distance on unscaled variables with different units;
- splitting the data randomly when groups, time or production lots should remain separated;
- interpreting a high correlation as a causal explanation;
- removing outliers without a domain analysis;
- choosing visualizations that hide the structure of the distribution;
- using a similarity measure that does not match the attribute types.

2 Data pre-processing – lesson 29/09/2026

Main tasks we need to perform in pre-processing:

- **Cleaning:** remove NaN values and noise, look for outliers or inconsistencies and deal with them.
- **Integration:** combine data sources to resolve inconsistencies or conflicts.
- **Reduction:** keep a smaller representation, without losing generality.
- **Transformation:** scale, aggregate or construct features.

After this steps the data are ready for the models.

The tasks are interconnected and the pipeline is not strictly sequential: for example cleaning often comes back after integration, because merging different sources can create new inconsistencies and redundancies.

Pre-processing matters because the same raw table can lead to different models depending on how we represent it:

- **Algorithms:** clustering, outlier detection and nearest neighbors are based on distances, so an unscaled or noisy attribute can dominate the result.
- **Databases:** data usually come from multiple sources, which can name the same attribute differently (`cust_id` vs. `customer_id`) or the same person differently (Bill vs. William).
- **Decisions:** pre-processing reduces storage and computation, and makes the results of the mining more interpretable and robust.

2.1 Data quality

Data have quality if they satisfy the requirements of the intended use, so quality is not an absolute property: the same database can be good enough for one user and inaccurate for another one. Bad data can look clean if we check only one dimension, so we have to consider all of them:

- **Accuracy:** are the values correct, and are the measurements accurate enough for the task? Wrong values come from faulty instruments, human or computer errors at data entry, errors in transmission, inconsistent formats or naming conventions. Errors in the unit of measure can be more harmful than missing values.
- **Completeness:** are all the relevant values recorded? Values can be unavailable, not entered because they were not considered important at the time, or deleted because they were inconsistent with other records.
- **Timeliness:** are the updates timely for the intended analysis? Data that will be correct only after late corrections arrive are of low quality in the meantime.
- **Consistency:** do the records obey the same codes and rules?
- **Believability:** how much the data are trusted by the users.
- **Interpretability:** how easily the data are understood by analysts and algorithms.

Remark. A particular source of inaccuracy is the *disguised missing data*: when a field is mandatory, users that do not want to give the information leave the default value (for example "January 1" as birthday). The value is formally present, but it is actually a missing value.

2.2 Data cleaning

It's very frequent to have missing information, noise or inconsistencies, especially if we have redundancy in the dataset. These problems are not exceptions, they are the normal case in real-world data, which can be:

- **Incomplete:** attribute values are lacking, relevant attributes are lacking, or only aggregate data are available (e.g. `Occupation` left blank).
- **Noisy:** the data contain random errors or outliers, due to faulty instruments or to data entry and transmission problems (e.g. `Salary = -10`).
- **Inconsistent:** codes or records are in conflict (e.g. `Age = 42` but `Birthday = 03/07/2010`, ratings that changed from 1,2,3 to A,B,C, duplicate records that disagree).

2.2.1 Missing values

- **Remove the rows with missing values.** If the number of missing values is small, this does not affect the overall analysis.
- **Manual filling.** If the number of missing values is small, we can fill them in manually.
- **Replace with a global constant.** Each missing value is replaced with the same constant (not ideal).
- **Replace with the mean or median of the column.** Better than a constant.
- **Replace with the mean or median of the same class.** We use only the samples that belong to the same class as the given tuple: for example, if we classify customers by credit risk, a missing income is replaced with the mean income of the customers in the same risk category. Smarter than the global mean.

- **Predict the missing values.** Build a model that uses the other features as input (the best option, but more complex).

Usually the most used solution is to replace the missing values with the mean or median of the column, but only if we have few missing values.

Some caveats on these methods:

- **Removing the tuple** is usually done when the class label is missing (in a classification task). It is not very effective unless the tuple has several missing attributes, because we throw away also the values that are present, and it is risky when the percentage of missing values varies a lot from attribute to attribute.
- **Manual filling** is accurate but tedious, and usually not feasible on a large dataset.
- **A global constant** (a label like "Unknown" or $-\infty$) can introduce an artifact: all the filled tuples share the same value, so the mining algorithm may think that they form an interesting concept or a new class.
- **Mean or median:** we use the mean when the distribution of the attribute is symmetric, and the median when it is skewed, since the mean is pulled by the extreme values.
- **The most probable value** can be obtained with regression, with inference based on a Bayesian formalism or with a decision tree built on the other attributes.

All the methods that fill in a value bias the data, because the filled-in value may not be the correct one. Predicting the value is the one that uses the most information from the data we have, so there is a greater chance to preserve the relationships between the attribute and the other ones.

Remark. A missing value is not always an error. For example, in a credit card application the field for the driver's license number is naturally left blank by who does not have a license. For this reason each attribute should have a rule for the *null* condition (are nulls allowed? how are they recorded?), with an explicit value like "not applicable".

2.2.2 Noise and smoothing

In real datasets, we can have noise, which is a random error or variance in the data. We can use smoothing techniques to remove noise from the data. The true signal change smoothly, while noise appears as rapid random changes. We can apply same smoothing techniques to remove noise from the data. Thera are some risk in smoothing, beaccause we can remove noise and at the same time distort the signal.

Rectangular smoothing filter:
we consider a window of size k(usually odd) and we replace the value of the point in the middle of the window with the average of the values in the window. Then we move the window of one position and we repeat the same operation. With this technique we can remove noise by reducing the variance between the samples. It is also called *unweighted sliding-average smooth*, and the size of the window is called *smooth width*. For example, with a smooth width of 3 points:

$$S_j = \frac{Y_{j-1} + Y_j + Y_{j+1}}{3}, \qquad j = 2, \ldots, n-1,$$

where S_j is the j-th point of the smoothed signal, Y_j is the j-th point of the original signal and n is the total number of points (the first and the last point do not have a complete window). The filter preserves the local value of the signal where the underlying signal is constant or linear,

because there the errors of the neighbors on the two sides compensate. Around peaks and valleys, instead, the neighbors have different values, so the average reduces the local maxima and raises the local minima.

Example 2.1 (Rectangular smoothing)**.**
Consider the signal $Y = (2, 4, 9, 4, 2)$, with a peak in the middle. With a smooth width of 3 we get

$$S_2 = \frac{2+4+9}{3} = 5, \qquad S_3 = \frac{4+9+4}{3} \approx 5.67, \qquad S_4 = \frac{9+4+2}{3} = 5,$$

so the peak is lowered from 9 to 5.67 and its neighbors are raised from 4 to 5. With a smooth width of 5 the peak becomes $S_3 = 21/5 = 4.2$: the wider the window, the more the peak is flattened. On a linear signal such as $Y = (1, 2, 3)$ we get instead $S_2 = 2 = Y_2$, the value is preserved.

The main parameter of this technique is the size of the window, the narrower the window the less smoothing we have, the wider the window the more smoothing we have. For example, on a noisy step signal an 11-point moving average still leaves some noise but keeps the edges of the step sharp, while a 51-point moving average removes almost all the noise but turns the edges into slow ramps.

Savitzky-Golay smoothing filter:
This technique is based on polynomial regression. We consider a window of size k and , instead of a plain average like in rectangular smoothing, we replace each data value by a linear combination of the values in the window:

$$S_j = \sum_{n=-n_l}^{n_r} c_n Y_{j+n},$$

Where n_l and n_r are the number of points to the left and right of the point j. For each point Y_j we least-squares fit a polynomial to alle the points in the window and set S_j to the value of the polynomial.

The formula above is the general form of the simplest digital filter (nonrecursive, or *finite impulse response*, filter), and the two filters differ only in the coefficients c_n. The rectangular filter is the special case in which all the coefficients are equal:

$$c_n = \frac{1}{n_l + n_r + 1}.$$

The average is the best estimate of the signal in the window if we approximate it with a constant. The idea of Savitzky-Golay is to approximate the underlying function in the window with a polynomial of higher order M, typically quadratic or quartic, so that the coefficients c_n preserve the higher moments of the signal (height and width of the peaks) and not only its mean.

Fitting a polynomial for every point would be laborious, but we do not need to do it. Least-squares fitting involves only a linear matrix inversion, so the coefficients of the fitted polynomial are linear in the data values. As a consequence the value of the fitted polynomial in j is a fixed linear combination of the samples in the window: the c_n depend only on the order M and on the window (n_l, n_r), not on the data, and they are computed once. Some examples (the coefficient of Y_j is in bold):

M	n_l	n_r	$c_{-n_l}, \ldots, c_{n_r}$
2	2	2	$-0.086,\ 0.343,\ \mathbf{0.486},\ 0.343,\ -0.086$
2	4	0	$0.086,\ -0.143,\ -0.086,\ 0.257,\ \mathbf{0.886}$
4	4	4	$0.035,\ -0.128,\ 0.070,\ 0.315,\ \mathbf{0.417},\ 0.315,\ 0.070,\ -0.128,\ 0.035$

The weights are not uniform: the central points count more and the points at the border of the window can have a negative weight. The window does not have to be symmetric (second row: only past values are used).

Example 2.2 (Savitzky-Golay vs. rectangular)**.**
On the signal $Y = (2, 4, 9, 4, 2)$ of the previous example, the filter with $M = 2$ and $n_l = n_r = 2$ gives

$$S_3 = -0.086 \cdot 2 + 0.343 \cdot 4 + 0.486 \cdot 9 + 0.343 \cdot 4 - 0.086 \cdot 2 \approx 6.77,$$

while the rectangular filter on the same 5 points gives 4.2. With the same window the Savitzky-Golay filter keeps the peak much closer to its original height.

The same happens on real signals: with a window of 33 points the sliding average strongly lowers and widens the narrow peaks, while a Savitzky-Golay filter of degree 4 with the same window follows their height and width much better (but its output is a bit less smooth).

In general the larger the smoothing window, the greater the noise reduction, but the greater the risk of distorting the signal. For peak-type signals, increasing the smoothing ratio(the ratio between the smoothing window size and the number of points in the half-width of the peak) improves the signal-to-noise ratio, but cause a reduction in amplitude. Together with the reduction in amplitude we have an increase in the bandwidth of the peak (the peak becomes lower and wider), and nearby peaks may merge. The optimum smooth width depends on the width and shape of the signal and on the digitization interval.

What matters is the smoothing ratio and not the size of the window alone. With windows of 7, 25 and 51 points, on a peak with a half-width of 80 points the ratios are $7/80 = 0.09$, $25/80 = 0.31$ and $51/80 = 0.64$ and the peak is only slightly lowered; on a peak with a half-width of 33 points the same windows give $7/33 = 0.21$, $25/33 = 0.76$ and $51/33 = 1.55$, and with the last one the peak is reduced to about half of its height.

How to choose the smoothing ratio depends on what we want to measure:

- **True peak height and width:** we should use smoothing ratios below 0.2.
- **Peak position** (the value on the x axis): we can use much larger smoothing ratios, because smoothing has no effect on the position of the peak, unless the peak becomes so wide that adjacent peaks overlap.

Smoothing is a modeling choice, we should apply it only when it supports the objective and not by default:

- **Yes, for visual inspection:** to make the broad structure easier to see and to show a signal without the distracting high-frequency noise.
- **Yes, before some algorithms:** when the later processing is harmed by high-frequency noise, e.g. detecting zero-crossings in derivatives or locating maxima, minima and inflection points.
- **No, before statistical fitting:** the accuracy of the least-squares parameters usually does not improve, the residuals become harder to interpret, the smoothed noise may look like real signal and the errors on the parameters may be underestimated.

Other ways of deal with noisy data:

- **Binning**: sort data, divide them in equal bins and smooth by bin mean, median or boundaries,

- **Regression**: fit a regression model to the data and use it to predict the value of the data points,
- **Inspection**: Clustering and human review to spot and remove outlier or suspicious values.

In **binning** each sorted value is smoothed by looking at its neighborhood, i.e. the values in the same bin, so it is a *local* smoothing. The bins can be *equal-frequency* (each bin contains the same number of values) or *equal-width* (each bin covers an interval of the same size). Then we can smooth by *bin means* (each value is replaced by the mean of its bin), by *bin medians*, or by *bin boundaries* (each value is replaced by the closest between the minimum and the maximum of its bin). The larger the bins, the greater the smoothing.

Example 2.3 (Binning)**.**
Sorted data for `price`: $4, 8, 15, 21, 21, 24, 25, 28, 34$, partitioned into equal-frequency bins of size 3.

	values	bin means	bin medians	bin boundaries
Bin 1	$4, 8, 15$	$9, 9, 9$	$8, 8, 8$	$4, 4, 15$
Bin 2	$21, 21, 24$	$22, 22, 22$	$21, 21, 21$	$21, 21, 24$
Bin 3	$25, 28, 34$	$29, 29, 29$	$28, 28, 28$	$25, 25, 34$

For example in bin 1 the mean is $(4 + 8 + 15)/3 = 9$, and with the boundaries 8 becomes 4 because it is closer to 4 than to 15.

In **regression** we conform the data to a function: with linear regression we find the best line that fits two attributes, so that one can be used to predict the other; with multiple linear regression we use more than two attributes and the data are fit to a multidimensional surface. For the **inspection**, clustering organizes similar values in groups, and the values that fall outside of all the clusters can be considered outliers; the automatic detection should be combined with checks based on the knowledge of the domain.

2.2.3 Data cleaning as a process

Cleaning is not a single operation: detection, correction and verification should be explicit steps, and they are iterated.

- **Detect discrepancies:** we use the *metadata* (the knowledge we already have about the data), such as the domain, the range, the dependencies and the distribution of each attribute. For example, for a distribution similar to a Gaussian the values that are more than two standard deviations away from the mean can be flagged as potential outliers.
- **Check rules:** the *uniqueness rule* says that each value of the attribute must be different from all the other values; the *consecutive rule* says that there are no missing values between the lowest and the highest value of the attribute (and all the values are unique, as in check numbers); the *null rule* specifies the use of blanks, question marks or other strings that indicate the null condition, and how these values have to be handled.
- **Scrub:** we use simple domain knowledge (spell-check, postal code checks) to detect errors and make corrections.
- **Audit:** we discover rules and relationships in the data and look for the values that violate them, e.g. with correlation analysis or with clustering to find outliers.
- **Verify:** we check whether the corrections created new inconsistencies.

The process of detecting and correcting has to be repeated, because a correction can introduce new discrepancies and some discrepancies are *nested*, they can be seen only after others have been fixed: a typo like "20010" in a year field may surface only once all the dates have been converted to the same format.

2.3 Data integration

Data integration is the merging of data from multiple data stores (databases, data cubes, flat files) into a coherent store. If it is done carefully it reduces redundancies and inconsistencies in the resulting dataset, and this improves the accuracy and the speed of the mining.

The main integration problems are:

- **Schema integration problem:** the same attribute can have different names in different sources (e.g. `A.cust-id` ≡ `B.cust-#`), so we have to integrate the metadata of the sources.
- **Entity identification problem:** we have to identify if two different objects in different databases are the same real-world entity.
- **Conflict resolution problem:** we have to resolve conflicts in attribute values for the same real-world entity.

Some details on each of them:

- To match schemas we use the metadata of each attribute: name, meaning, data type, range of the permitted values and null rules. They also tell us when the same information is coded differently (a payment type coded as "H" and "S" in one database and as 1 and 2 in another one).
- We have to pay attention also to the *structure* of the data, so that functional dependencies and referential constraints of the source match those of the target. For example in one system a discount is applied to the whole order, in another one to each line item of the order: if we do not notice it, the items in the target system are discounted in the wrong way.
- Entity identification requires matching that goes beyond the exact strings: Bill Clinton and William Clinton are the same person.
- The values of the same entity can differ because of different representation, scaling or encoding: a weight in metric units in one system and in British imperial units in another; the price of a hotel room in different currencies, and including different services and taxes; grades from A+ to F in one university and from 1 to 10 in another. The attributes can also be at different levels of abstraction: `total_sales` can refer to one branch in one database and to all the stores of a region in another one.

2.4 Redundancy

Redundant data can slow data mining. we can have redundancy not only with attribute with the same value but also if two attributes have the same trend and are **strongly correlated**. Redundancy does not only slow the mining, it also creates inconsistencies. It can come from:

- **Different names:** the same attribute or object appears with different names in the sources; object identification removes this duplicated meaning.
- **Derived data:** an attribute can be derived from another attribute or from a set of attributes, e.g. the annual revenue can be computed from the monthly revenue.

- **Duplicated tuples:** two or more tuples describe the same data entry. It often happens with denormalized tables (used to avoid joins), and the duplicates can become inconsistent when only some of the occurrences are updated: the same purchaser's name may appear with different addresses.

Redundant attributes can be detected statistically with *correlation analysis*, that measures how strongly one attribute implies the other: for nominal attributes we use the chi-square test, for numeric attributes the correlation coefficient and the covariance.

To understand if two attributes (only nominal attribute) are correlated we can use the **chi-square test**. The chi-square test is a statistical test that measures the association between two categorical variables by using the contingency table. The contingency table contains the observed frequencies o_{ij} of the combinations of the two variables. The chi-square statistic is calculated as:

$$X^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(o_{ij} - e_{ij})^2}{e_{ij}}$$

Where r is the number of rows, c is the number of columns, and e_{ij} is the expected frequency for cell (i, j) under the null hypothesis of independence. The expected frequency is calculated as:

$$e_{ij} = \frac{(count(A = a_i)) \times (count(B = b_j))}{n}$$

Where A and B are the two attributes we are testing for independence, a_i and b_j are the specific values of A and B. Here the r values of A are the rows and the c values of B are the columns of the table, n is the total number of tuples, and the sum is computed over all the $r \times c$ cells (the slides and the book call c the number of values of A and r the number of values of B, the formula is the same). The expected frequency comes directly from the assumption of independence: $e_{ij} = n \cdot P(A = a_i, B = b_j)$ and, since A and B are independent, the joint probability is the product of the two marginal probabilities, that we estimate with the relative frequencies:

$$e_{ij} = n \cdot P(A = a_i) \cdot P(B = b_j) = n \cdot \frac{count(A = a_i)}{n} \cdot \frac{count(B = b_j)}{n}.$$

So in every row (column) the sum of the expected frequencies is equal to the sum of the observed ones. The larger the value of X^2, the more likely the two variables are related, and the cells that contribute the most are those where the observed count is far from the expected one.

The chi-square is a type of probability distribution that is used in hypothesis testing to determine independence. **The null hypothesis** state that no association exists between the two variables, while **the alternative hypothesis** states that an association exists.

More precisely, under the null hypothesis H_0 (the two variables are statistically independent in the population) the statistic X^2 follows a χ^2 distribution (vedi stea). The **p-value** is the probability, under H_0, of obtaining a value at least as large as the observed one, i.e. the area under the curve of the distribution to the right of the computed X^2.

There is not a single χ^2 distribution, the one we have to use depends on the **degrees of freedom**: the number of cells of the contingency table that can vary freely, given the row and column totals. For a table with r rows and c columns

$$df = (r - 1)(c - 1).$$

For example, in a 2×2 table with fixed totals, once we choose one cell the other three are determined ($df = 1$); in a 2×3 table we can choose two cells ($df = 2$). All the χ^2 distributions are asymmetric and skewed to the right, and their shape changes with the degrees of freedom.

To decide we fix a **significance level** (a small value such as 0.05, 0.01 or 0.001: the lower the significance level, the higher the confidence in the decision). We reject the null hypothesis

when the computed X^2 is higher than the value in the table of the χ^2 distribution for our degrees of freedom and significance level, or equivalently when the p-value is lower than the significance level. If we reject H_0 we say that A and B are statistically correlated. Some values of the table (probability of a larger value of X^2):

df	0.10	0.05	0.01
1	2.71	3.84	6.63
2	4.61	5.99	9.21
3	6.25	7.81	11.34
4	7.78	9.49	13.28
5	9.24	11.07	15.09

For example with $X^2 = 5$ and $df = 4$ the p-value is about 0.3: we are below 7.78, so we cannot reject independence.

Example 2.4 (Chi-square on a 2×2 table)**.**
In a group of 1500 people we record if they like science fiction and if they play chess. The expected frequencies are in parentheses.

	Play chess	Not play chess	Sum
Like science fiction	250 (90)	200 (360)	450
Not like science fiction	50 (210)	1000 (840)	1050
Sum	300	1200	1500

For example $e_{11} = \frac{450 \times 300}{1500} = 90$. Then

$$X^2 = \frac{(250-90)^2}{90} + \frac{(50-210)^2}{210} + \frac{(200-360)^2}{360} + \frac{(1000-840)^2}{840}$$
$$= 284.44 + 121.90 + 71.11 + 30.48 = 507.93.$$

The degrees of freedom are $(2-1)(2-1) = 1$. With $df = 1$ the value needed to reject independence at the 0.001 significance level is 10.828: our value is far above it, so we reject H_0 and conclude that the two attributes are correlated in this group.

Example 2.5 (Chi-square on a larger table)**.**
We want to know if education level and marital status are related, using a simple random sample of $n = 300$ people.

	Middle school	High school	Bachelor's	Master's	PhD	Total
Never married	18	36	21	9	6	90
Married	12	36	45	36	21	150
Divorced	6	9	9	3	3	30
Widowed	3	9	9	6	3	30
Total	39	90	84	54	33	300

The expected frequencies are computed from the totals, e.g. for (never married, middle school) $e_{11} = \frac{90 \times 39}{300} = 11.7$ and for (never married, high school) $e_{12} = \frac{90 \times 90}{300} = 27$. Summing over the 20 cells:

$$X^2 = \frac{(18-11.7)^2}{11.7} + \frac{(36-27)^2}{27} + \cdots = 23.57.$$

The degrees of freedom are $(4-1)(5-1) = 12$, and for the χ^2 distribution with $df = 12$ we have $P(\chi^2 > 23.6) = 0.023$. The p-value is lower than 0.05, so at this significance level we

reject H_0: marital status and education are related in the population. With a stricter level such as 0.01 we could not reject it.

Remark. Correlation is not causality: the test tells us that two attributes are related, not that one causes the other. The number of hospitals and the number of car thefts in a city are correlated, but only because both are causally linked to a third variable, the population.

3 Lesson 03 — 06/10/2026

3.1 Correlation Analysis with numeric data

The `correlation coefficient`, also called `Pearson's product-moment coefficient`, is defined as

$$r_{A,B} = \frac{\sum_{i=1}^{n}(a_i - \bar{A})(b_i - \bar{B})}{n\sigma_A\sigma_B}$$

where n is the number of tuples, $\bar{A}$ and $\bar{B}$ are the mean values of the attributes A and B. σ_A and σ_B are the standard deviations of the attributes A and B.

- If $r_{A,B} > 0$, then A and B are positively linearly correlated.
- If $r_{A,B} < 0$, then A and B are negatively linearly correlated.
- If $r_{A,B} = 0$, then there is no linear correlation between A and B (this does not mean that they are independent, see the remark below).

For visualization of the correlation between two attributes we can use a scatter plot and see the trend of the points.

The same coefficient can be computed without centering the data first, using the sum of the cross-products $a_i b_i$ (for each tuple we multiply the value of A by the value of B):

$$r_{A,B} = \frac{\sum_{i=1}^{n}(a_i b_i) - n\bar{A}\bar{B}}{n\sigma_A\sigma_B}, \qquad -1 \leq r_{A,B} \leq +1$$

The closer $|r_{A,B}|$ is to 1, the stronger the linear relation between the two attributes. This is why we compute it during data integration: a high value tells us that one of the two attributes carries almost the same information as the other and may be removed as redundant.

Remark. We need to be careful with the case $r_{A,B} = 0$. The Pearson coefficient only detects *linear* relationships: if the points lie on a parabola, B is completely determined by A and still $r_{A,B} = 0$. So $r_{A,B} = 0$ means that there is no linear correlation between A and B. If A and B are independent then $r_{A,B} = 0$, but the converse is not true in general: it holds only under additional assumptions, for example when the data follow a multivariate normal distribution.

Also, correlation does not imply causality. For example the number of hospitals and the number of car thefts in a region are correlated, but neither causes the other: both depend on a third attribute, the population.

3.1.1 Covariance

Covariance is a measure similar to correlation: both tell us how much two numeric attributes change together.

Definition 3.1 (Covariance)**.**
Given two numeric attributes A and B and n observations $(a_1, b_1), \ldots, (a_n, b_n)$, with mean

(expected) values $\bar{A} = \mathbb{E}(A)$ and $\bar{B} = \mathbb{E}(B)$, the covariance between A and B is

$$\mathrm{Cov}(A, B) = \mathbb{E}\big((A - \bar{A})(B - \bar{B})\big) = \frac{\sum_{i=1}^{n}(a_i - \bar{A})(b_i - \bar{B})}{n}$$

It can also be computed as

$$\mathrm{Cov}(A, B) = \mathbb{E}(A \cdot B) - \bar{A}\bar{B}$$

which is often easier to use in the calculations.

The correlation coefficient is the covariance divided by the two standard deviations, so it is a covariance that does not depend on the scale of the attributes:

$$r_{A,B} = \frac{\mathrm{Cov}(A, B)}{\sigma_A \sigma_B}$$

- $\mathrm{Cov}(A, B) > 0$: A and B tend to be above (or below) their expected values together.
- $\mathrm{Cov}(A, B) < 0$: when one attribute is above its expected value, the other tends to be below.
- $\mathrm{Cov}(A, B) = 0$: this does not always imply independence. If A and B are independent then $\mathbb{E}(A \cdot B) = \mathbb{E}(A) \cdot \mathbb{E}(B)$ and the covariance is 0, but there are pairs of attributes with covariance 0 that are not independent.

The variance is the special case of the covariance of an attribute with itself: $\mathrm{Var}(A) = \mathrm{Cov}(A, A)$.

Example 3.2 (Covariance of two stock prices)**.**
We observe the prices of two stocks at five time points: $A = (6, 5, 4, 3, 2)$ and $B = (20, 10, 14, 5, 5)$. We want to know if they rise and fall together.

$$\mathbb{E}(A) = \frac{6 + 5 + 4 + 3 + 2}{5} = 4, \qquad \mathbb{E}(B) = \frac{20 + 10 + 14 + 5 + 5}{5} = 10.8$$

$$\mathrm{Cov}(A, B) = \frac{6 \cdot 20 + 5 \cdot 10 + 4 \cdot 14 + 3 \cdot 5 + 2 \cdot 5}{5} - 4 \cdot 10.8 = 50.2 - 43.2 = 7$$

The covariance is positive, so the two prices tend to rise together.

3.2 Data reduction

To reduce the data size we can work in different ways:

- `Dimensionality reduction`: reduce the number of attributes, for example by removing redundant or irrelevant attributes.
- `Numerosity reduction`: reduce the number of tuples, for example by sampling or aggregation, or by replacing the objects with models.
- `Data compression`: reduce the data size by encoding the data more efficiently.

Why should we reduce the number of dimensions? We need to avoid the `curse of dimensionality`, a phenomenon that occurs when the number of dimensions increases and that has three effects:

- `Sparsity`: the data become increasingly sparse and the density becomes less meaningful.
- `Distances`: the distance between points becomes less meaningful, because all the points appear far from each other when the space is too sparse. This is a problem because almost all the algorithms in machine learning are based on distance or density.

- `Combinations`: the possible combinations of subspaces grow exponentially with the number of dimensions, so visualization becomes difficult and the time and space requirements increase.

Besides avoiding the curse of dimensionality, reducing the number of dimensions helps us to eliminate irrelevant features, reduce noise, reduce the time and space required by the mining algorithms and make visualization easier.

To reduce dimensionality we see two different approaches:

- `Linear projection methods` (e.g. PCA, Principal Component Analysis): we transform the data and then work with the transformed data.
- `Heuristic methods`: we use greedy methods directly on the data source.

3.3 Principal Component Analysis (PCA)

In PCA we want to find the directions of maximum variance in the data and project the data onto these directions.

- Step1: Calculate Adjusted Data Matrix: we center the data by subtracting the mean of each attribute from the data points.
- Step2: Calculate Covariance Matrix from adjusted Data.
- Step3: Calculate Eigenvalues and Eigenvectors of the covariance matrix. If the eigenvalues are 0 or really small we can discard them and their corresponding eigenvectors.
- Step4: Transform the data onto the new basis formed by the relevant eigenvectors. $F = E^T A$ where F is the transformed data set, E is the matrix of eigenvectors and A is the adjusted data matrix.

More precisely, we are given N data vectors in f dimensions and we look for $k \leq f$ orthogonal vectors, the *principal components*, that can best be used to represent the data. We see the steps again with the formulas.

- Before starting we normalize the input data, so that all the attributes are on comparable ranges. Otherwise the attributes with a large domain have the largest variance and dominate the others. PCA works only with numeric data.
- Step1: we write the data set as a matrix D with f rows (dimensions) and N columns (data samples). M is the vector of the means, where M_i is the mean of the values in dimension i. The adjusted data set is $A = D - M$, where M_i is subtracted from all the values of row i. Now every dimension of A has mean 0.
- Step2: the covariance matrix C is a $f \times f$ matrix where C_{ij} is the covariance between dimension i and dimension j (on the diagonal we find the variances):

$$\text{Cov}(X, Y) = \frac{\sum_{i=1}^{N}(X_i - \bar{X})(Y_i - \bar{Y})}{N - 1}$$

  Since the means of the dimensions in A are 0, we can compute the whole matrix at once:

$$C = \frac{AA^T}{N - 1}$$

- Step3: the eigenvectors of C are the columns of the matrix E and they are the axes of the new space. Each eigenvalue is the variance of the data along the corresponding eigenvector, so we sort the components by decreasing eigenvalue: the first axis shows the most variance, the second axis the next highest variance, and so on. Discarding the components with the smallest eigenvalues reduces the dimensionality of the new basis and we lose little information.

- Step4: if we keep only k eigenvectors in E, then $F = E^T A$ has k rows instead of f: the dimensions of the new data set are less than the ones of A. To recover A from F we use the fact that E is orthogonal, so $E^{-1} = E^T$:

$$(E^T)^{-1}F = (E^T)^{-1}E^T A \quad \Longrightarrow \quad EF = A$$

  If we discarded some components, EF is only an approximation of A, but a good one if the discarded components were weak.

Remark. In the covariance matrix of PCA the slides divide by $N-1$, while in the definition of covariance seen before we divide by n. For a large number of samples the difference is negligible.

Example 3.3 (Covariance matrix for PCA)**.**
We have $N = 5$ people, each one described by Height, test Score and Age, so $f = 3$.

	S1	S2	S3	S4	S5	Mean
Height	64	66	68	69	73	68
Score	580	570	590	660	600	600
Age	29	33	37	46	55	40

We subtract the mean from each row and compute variances and covariances, for example

$$\text{Var(Height)} = \frac{(-4)^2 + (-2)^2 + 0^2 + 1^2 + 5^2}{5-1} = \frac{46}{4} = 11.5$$

$$\text{Cov(Height, Score)} = \frac{(-4)(-20) + (-2)(-30) + 0 \cdot (-10) + 1 \cdot 60 + 5 \cdot 0}{5-1} = \frac{200}{4} = 50$$

Repeating the computation for all the pairs we obtain the covariance matrix

$$C = \begin{pmatrix} 11.5 & 50 & 34.75 \\ 50 & 1250 & 205 \\ 34.75 & 205 & 110 \end{pmatrix}$$

where rows and columns are in the order Height, Score, Age. The variance of Score is much larger than the others only because of its scale: this is why we should normalize the data before applying PCA.

PCA is an unsupervised approach, because we do not use any information about the class labels of the data points. PCA is also a transformation method, because we transform the data into a new basis formed by the eigenvectors that are different features from the original ones.

This is the main difference with subset selection, that we see next: subset selection keeps some of the original attributes, PCA instead combines all of them to create a smaller set of new variables. Each principal component is a linear combination of the original attributes, so the new features are harder to interpret, and PCA is a linear method: it works well when the structure of the data is approximately linear, it becomes ineffective when it is not.

3.4 Subset selection

A second way to reduce dimensionality is to select a subset from the original features. We need to select only the features relevant to our task, trying to avoid redundancy and irrelevance. We can't try every possible subset of features to find the best one, we need to use a greedy approach.

We will see two different approaches to select a subset of features:

- `Forward selection`: we start with an empty set of features and we add one feature at a time, selecting the most relevant and least redundant feature at each step.
- `Backward elimination`: we start with the full set of features and we remove one feature at a time, removing the least relevant and most redundant feature at each step.

For both approaches we need a metric to define relevancy and redundancy.

In detail, the attributes we want to remove are of two kinds:

- Redundant attributes: they duplicate much or all of the information contained in other attributes. For example the purchase price of a product and the sales tax paid.
- Irrelevant attributes: they contain no information useful for the mining task. For example the student ID when we want to predict the GPA.

Keeping irrelevant attributes (or leaving out relevant ones) can confuse the mining algorithm and give patterns of poor quality, and the extra volume slows down the mining. Working on fewer attributes also makes the discovered patterns easier to understand.

With d attributes there are 2^d possible subsets, this is why an exhaustive search is not feasible. A greedy method always makes the choice that looks best at the moment (a locally optimal choice), hoping that this leads to a globally good solution. It is not guaranteed to find the optimal subset, but in practice it works well.

Example 3.4 (Forward selection and backward elimination)**.**
The initial set of attributes is $\{A_1, A_2, A_3, A_4, A_5, A_6\}$.
Forward selection: $\{\} \to \{A_1\} \to \{A_1, A_4\} \to \{A_1, A_4, A_6\}$. At each iteration we test the remaining features and we add the one with the best score.
Backward elimination: $\{A_1, A_2, A_3, A_4, A_5, A_6\} \to \{A_1, A_3, A_4, A_5, A_6\} \to \{A_1, A_4, A_5, A_6\} \to \{A_1, A_4, A_6\}$. At each iteration we discard the feature whose removal hurts least.

A good subset is not just made of individually strong features: each feature should add information without duplicating what is already selected. A typical way to score a candidate feature A_j given the set S of the features already selected is

$$\text{score}(A_j \mid S) = \text{relevance}(A_j, \text{target}) - \text{redundancy}(A_j, S)$$

In forward selection we add the feature with the maximum score, in backward elimination we remove the feature with the lowest marginal contribution. The procedure stops when a stopping criterion is met, for example a threshold on the measure or a fixed number of features.

Other heuristic methods are:

- Combination of forward selection and backward elimination: at each step we select the best attribute and we remove the worst among the remaining ones.
- Decision tree induction: we build a decision tree on the data; the attributes that do not appear in the tree are assumed to be irrelevant, the ones that appear form the reduced subset.
- The best and worst attributes can also be chosen with tests of statistical significance (that assume the attributes independent of one another) or with measures like the information gain. The slides also name branch and bound among the search strategies.

3.4.1 An example of heuristic: NMIFS

An example of forward selection is NMIFS (Normalized Mutual Information Feature Selection, by Estévez, Tesmer, Perez and Zurada, 2009). We need to be careful: this method needs the ground truth, because the relevance of a feature is measured with respect to the class labels.

Definition 3.5 (Mutual information)**.**
Given two discrete variables X and Y with alphabets $\mathcal{X}$ and $\mathcal{Y}$ (the sets of their possible values), joint probability mass function $p(x, y)$ and marginal probabilities $p(x)$ and $p(y)$, the mutual information between X and Y is

$$I(X, Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \cdot \log \frac{p(x, y)}{p(x)p(y)}$$

If X and Y are independent then $p(x, y) = p(x) \cdot p(y)$, every logarithm is 0 and so $I(X, Y) = 0$. The sum runs over all the possible combinations of values: for example if X is the age with values {young, medium, old} and Y is the output with values {class1, class2, class3}, we need the nine probabilities $p(\text{young}, \text{class1})$, $p(\text{young}, \text{class2})$, $\ldots$, $p(\text{old}, \text{class3})$.

The problem is: given an initial set F with n features, find the subset $S \subset F$ with k features that maximizes the mutual information $I(C; S)$ between the class variable C and the selected features. NMIFS uses the mutual information for both terms of the score:

- Relevance: $I(C; f_i)$, the mutual information between the class and the feature f_i.
- Redundancy: the average normalized mutual information between f_i and the features already selected, where

  $$NI(f_i; f_s) = \frac{I(f_i; f_s)}{\min\{H(f_i), H(f_s)\}}$$

  and H is the entropy. The normalization compensates for the bias of the mutual information toward features with many values and restricts the values to the range $[0, 1]$.

At each step we select the feature that maximizes

$$G = I(C; f_i) - \frac{1}{|S|} \sum_{f_s \in S} NI(f_i; f_s)$$

The algorithm is:

1. Initialization: $F = \{f_i \mid i = 1, \ldots, N\}$ is the initial set of N features and $S = \emptyset$.
2. Compute the mutual information with respect to the classes, $I(f_i; C)$, for each $f_i \in F$.
3. Select the first feature $\hat{f}_i = \arg\max_{i=1,\ldots,N} I(f_i; C)$. Set $F \leftarrow F \setminus \{\hat{f}_i\}$ and $S \leftarrow \{\hat{f}_i\}$.
4. Greedy selection, repeated until $|S| = k$:
   (a) compute the mutual information between features, $I(f_i; f_s)$, for all the pairs with $f_i \in F$ and $f_s \in S$;
   (b) select the feature $\hat{f}_i \in F$ that maximizes G. Set $F \leftarrow F \setminus \{\hat{f}_i\}$ and $S \leftarrow S \cup \{\hat{f}_i\}$.
5. Output the set S of the selected features.

3.5 Attribute creation / feature generation

Sometimes the features we need are not in the original data, so we need to create new ones. We can create features in three different ways:

- `Extraction`: transformation specific to the domain, for example we can extract features from images or text, or map the data to a new space.
- `Construction`: we can combine existing features to create new ones, for example a new feature that is the sum of two existing features.
- `Discretization`: we can convert continuous features into discrete ones.

Typical examples of mapping the data to a new space are the Fourier transformation and the wavelet transformation. For construction, a classic example is adding the attribute *area* computed from the attributes *height* and *width*: the new attribute makes explicit a relation between the original ones that the mining algorithm may not find alone, and it is still interpretable. Discretization, that we see at the end of this lesson, also reduces the data size and prepares the data for the methods that work with categories.

Remark. The discrete wavelet transform (DWT) can also be used to reduce the data. It transforms a data vector into a vector of wavelet coefficients of the same length, but we can store only the strongest coefficients (for example the ones above a threshold) and set the others to 0. We obtain a sparse, compressed approximation of the data, that can be rebuilt applying the inverse transform. Compared to the Fourier transform, with the same number of coefficients the DWT usually gives a more accurate approximation, and wavelets are localized in space so they preserve local details.

3.6 Numerosity reduction

We can reduce the number of stored values by choosing optimal representation. We need to be careful because we can lose information: smaller data often means higher speed and lower cost, but also lower accuracy. We need to preserve the data structure and the relevant information. We can use two families of methods:

- Parametric methods: We assume that the data fit a models and we compute, and only store, the parameters of the model. For example, we can use linear regression to fit a line to the data and use the parameters of the line to represent the data.
- Non-parametric methods: we do not assume any specific distribution for the data, but we use techniques that are based on the data itself to find families or aggregations. For example, we can use histograms or kernel density estimation to model the data.

There is a trade-off between compression and detail: smaller representations can be faster, but they may hide local structure. The right method depends on the query and on the mining task.

3.6.1 Parametric methods

With regression we replace many observations with a fitted function: we estimate the parameters of the model, we store only them and we discard the raw data (except possible outliers).

`Linear regression:` the data are modeled to fit a straight line

$$y = w_1 x + w_0$$

The two regression coefficients w_1 (slope) and w_0 (intercept) define the line and are estimated from the data with the least squares criterion, that is we choose the line that minimizes the sum

of the squared errors between the known values y_i and the values predicted by the line. If $|D|$ is the size of the data set and $\bar{x}$, $\bar{y}$ are the means:

$$w_1 = \frac{\sum_{i=1}^{|D|}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|}(x_i - \bar{x})^2}, \qquad w_0 = \bar{y} - w_1\bar{x}$$

Example 3.6 (Linear regression)**.**
For five students x_i is the score on the aptitude test and y_i is the statistics grade.

Student	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})^2$	$(x_i - \bar{x})(y_i - \bar{y})$
1	95	85	17	8	289	136
2	85	95	7	18	49	126
3	80	70	2	−7	4	−14
4	70	65	−8	−12	64	96
5	60	70	−18	−7	324	126
Sum	390	385			730	470
Mean	78	77				

$$w_1 = \frac{470}{730} = 0.6438, \qquad w_0 = 77 - 0.6438 \cdot 78 = 26.78$$

The regression equation is $y = 0.6438x + 26.78$: instead of the five pairs we store only two numbers.

`Multiple linear regression:` the response variable is modeled as a linear function of a multidimensional feature vector, for example with two attributes

$$y = w_0 + w_1x_1 + w_2x_2$$

The coefficients are still found with least squares. In matrix form, if X is the matrix of the observations (with a first column of 1s for the intercept w_0) and Y the vector of the responses:

$$W = (X^TX)^{-1}X^TY$$

`Non-linear regression:` many non-linear functions can be transformed into the linear form above. For example the polynomial regression

$$y = w_0 + w_1x + w_2x^2 + w_3x^3$$

becomes linear if we define the new variables $x_1 = x$, $x_2 = x^2$, $x_3 = x^3$:

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3$$

and it can be solved with the methods of multiple regression.

`Log-linear models:` they approximate discrete multidimensional probability distributions. They are useful for smoothing and for reduction.

3.6.2 Non-parametric methods

- Histograms: we divide the data into buckets and we store the frequency of each bucket. We can use equi-width or equi-depth histograms.
- Clustering: we group the data into clusters, by similarity, and we store the cluster centroids. We can use k-means or hierarchical clustering.

- Sampling: we select a subset of the data and we store only the selected data points. We can use random sampling or stratified sampling.

Some details on the three families.

`Histograms:` a histogram partitions the values of an attribute into disjoint ranges called buckets (or bins). For each bucket we store only a summary, like the count, the average or the sum of its values. If a bucket represents a single value with its frequency it is called a singleton bucket (useful to store high-frequency outliers); to reduce the data more, each bucket represents a continuous range. In an equi-width histogram all the buckets have the same range width, in an equi-depth (equal-frequency) histogram each bucket contains roughly the same number of samples.

`Clustering:` for each cluster we store only its representation, the centroid and the diameter. It is effective if the data are really organized in clusters, it is less effective if the data are smeared.

`Sampling:` we represent the whole data set of N tuples with a small sample of s tuples. The advantage is that the cost of obtaining the sample is proportional to s and not to N, so a mining algorithm can run with a complexity that is potentially sub-linear in the size of the data, while other reduction techniques need at least one complete pass through the data. Simple random sampling may fail if the data are skewed, stratified sampling improves the representation.

We can have sampling with replacement or without replacement. The common ways to sample a data set D of N tuples are:

- Simple random sample without replacement (SRSWOR) of size s: we draw s tuples from D, every tuple has the same probability to be chosen, and a drawn tuple is removed from D, so it can not be drawn again.

- Simple random sample with replacement (SRSWR) of size s: as before, but after a tuple is drawn it is recorded and then placed back in D, so it may be drawn again.

- Cluster sample: if the tuples of D are grouped into M mutually disjoint clusters, we take a simple random sample of s clusters, with $s < M$. For example tuples in a database are retrieved a page at a time, so each page can be a cluster.

- Stratified sample: D is divided into mutually disjoint parts called strata and we draw a sample from each stratum, proportionally to its size. It is useful when the data are skewed, because also the small groups are still represented. For example, with one stratum for each age group of the customers, the group with the fewest customers is sure to be represented.

The key difference between sampling with and without replacement is whether the draws are independent. Consider a finite population of size N with variance $\sigma^2 = \frac{1}{N}\sum_i (x_i - \mu)^2$ and let X_1, X_2 be the first and the second draw.

- With replacement: after the first draw the population is restored, so the first draw does not affect the probabilities of the second one. X_1 and X_2 are independent and $\mathrm{Cov}(X_1, X_2) = 0$.

- Without replacement: after the first draw one value has been removed, so the first draw changes what can happen in the second. X_1 and X_2 are negatively dependent:

$$\mathrm{Cov}(X_1, X_2) = -\frac{\sigma^2}{N-1}$$

  Intuitively, if the first value is high, fewer high values remain. The covariance is small when N is large.

3.6.3 Aggregation and compression

- Data cube aggregation: also aggregation is a way to reduce the data. The base cuboid stores the data at the lowest level, and we can aggregate at multiple levels: for example from the sales per quarter we compute the sales per year. We use the smallest representation that is sufficient for the task, and we answer the aggregate queries with the cube when possible.
- Lossless compression: the original data can be reconstructed from the compressed data without any loss of information. It is typical of string compression, where there are well-tuned theories and algorithms, but only a limited manipulation of the data is possible without expanding it.
- Lossy compression: we can reconstruct only an approximation of the original data. It is typical of audio and video compression, often with progressive refinement, and sometimes small fragments of the signal can be reconstructed without rebuilding the whole. Time sequences are not like audio: they vary slowly with time.

Dimensionality reduction and numerosity reduction can also be considered forms of data compression.

3.7 Transformation

The last step in data pre-processing is transformation, which is the process of changing the format, structure or values of the data. We have three different families of transformations:

- `Smoothing`: removing noise.
- `Aggregation & construction`: create new attributes from existing ones.
- `Normalization & discretization`: scale values to a smaller range.

`Data normalization`. These techniques are applied to ensure a more efficient and accurate data analysis. Normalization brings all the values into a specific range, to ensure that an attribute isn't more important just because of its higher scale.

We can see the problem in the computation of the Euclidean distance between two points p and q with n attributes:

$$d(p,q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

If the first attribute has range 0–1000000 and the others have range 0–1, the first term is predominant and the distance depends practically only on it. The same happens when we change the unit of measure: expressing an attribute in smaller units gives it a larger range and so a larger weight. For this reason normalization is needed by the methods based on distances (nearest-neighbor classification, clustering), and it also speeds up the learning phase of neural networks.

`min-max normalization:` we rescale the data to a specific range, usually [0,1]. We can use the formula:

$$v' = \frac{v - v_{min}}{v_{max} - v_{min}}(new_max - new_min) + new_min$$

where v_{min} and v_{max} are the minimum and maximum values of the attribute and [new_min, new_max] is the new range. It is a linear transformation, so it preserves the relationships among the original values. The problem is that if a future value falls outside the original range $[v_{min}, v_{max}]$, it is mapped outside the new range (an "out-of-bounds" error).

Example 3.7 (Min-max normalization)**.**
The income ranges from \$12,000 to \$98,000 and we normalize it to $[0.0, 1.0]$. The value \$73,600 is mapped to

$$\frac{73{,}600 - 12{,}000}{98{,}000 - 12{,}000}(1.0 - 0) + 0 = 0.716$$

`z-score transformation`: Use mean and standard deviation and the distance increase using number of standard deviations as scale.

$$v' = \frac{v - \mu_A}{\sigma_A}$$

This normalization works well with outliers, which are instead a problem for min-max. It is also the one to use when the actual minimum and maximum of the attribute are unknown.

Example 3.8 (z-score normalization)**.**
With $\mu_A = 54{,}000$ and $\sigma_A = 16{,}000$ the income \$73,600 is transformed to

$$\frac{73{,}600 - 54{,}000}{16{,}000} = 1.225$$

A variation replaces the standard deviation with the mean absolute deviation of A

$$s_A = \frac{1}{n}\big(|v_1 - \mu_A| + |v_2 - \mu_A| + \cdots + |v_n - \mu_A|\big), \qquad v' = \frac{v - \mu_A}{s_A}$$

that is more robust to outliers than σ_A, because the deviations from the mean are not squared.

`Decimal scaling`: we move the comma in the number depending on the max absolute values of the attribute.

$$v' = \frac{v}{10^j}$$

where j is the smallest integer such that $\max(|v'|) < 1$.

Example 3.9 (Decimal scaling)**.**
The values of A range from -986 to 917, so the maximum absolute value is 986. We divide each value by 1000 ($j = 3$): -986 becomes -0.986 and 917 becomes 0.917.

Remark. Normalization can change the original data quite a bit, especially with z-score and decimal scaling. We also need to save the parameters used (for example mean and standard deviation for the z-score), so that future data can be normalized in the same way.

3.7.1 Discretization

With discretization we divide the range of a continuous attribute into intervals, and the interval labels (for example 0–10, 11–20, or conceptual labels like *youth*, *adult*, *senior*) replace the actual values. We do it to reduce the data size and to prepare the data for classification, because some algorithms work only with categorical attributes. We recall the types of attributes: nominal (unordered categories, like the color), ordinal (ordered categories, like the rank) and numeric (integer or real numbers).

The methods differ in two aspects:

- Supervised vs. unsupervised: a method is supervised if it uses the class labels to choose the intervals, unsupervised otherwise.
- Top-down (splitting) vs. bottom-up (merging): a top-down method first finds one or a few split points (cut points) that divide the whole range, and then repeats recursively on the

resulting intervals. A bottom-up method starts by considering all the values as potential split points and removes some of them by merging neighboring values into intervals, again recursively.

The typical methods are binning and histogram analysis (unsupervised, top-down split), clustering analysis (unsupervised, top-down or bottom-up), correlation / χ^2 analysis (supervised, bottom-up merge) and the entropy-based methods (supervised, top-down split).

`Unsupervised discretization: binning.` We choose the number N of intervals and one of two partitioning rules:

- Equal-width partitioning: we divide the range into N intervals of equal size (a uniform grid). If A and B are the lowest and the highest values of the attribute, the width of the intervals is
$$W = \frac{B - A}{N}$$
It is the most straightforward method, but outliers may dominate the presentation and skewed data are not handled well.
- Equal-depth (equal-frequency) partitioning: we divide the range into N intervals, each containing approximately the same number of samples. It scales well with the data and handles skewed data better.

Binning does not use the class information and it is sensitive to the number of bins chosen by the user and to the presence of outliers. Histogram analysis works in the same way, with the buckets seen for numerosity reduction. Both can be applied recursively to the resulting partitions to generate a hierarchy of intervals.

Example 3.10 (Binning)**.**
Sorted prices: 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34. Binning can also be used to smooth the sorted values.

Method	Bin 1	Bin 2	Bin 3
Equal-frequency bins	4, 8, 9, 15	21, 21, 24, 25	26, 28, 29, 34
Smooth by bin means	9, 9, 9, 9	23, 23, 23, 23	29, 29, 29, 29
Smooth by boundaries	4, 4, 4, 15	21, 21, 25, 25	26, 26, 26, 34

`Clustering.` We can also discretize by clustering the values of the attribute and using one interval for each cluster. Clustering looks at the distribution of the data, so for example k-means may give better intervals than naive binning: in the comparison on the slides equal-width and equal-frequency binning cut through the natural groups of points, while k-means separates them.

`Supervised discretization.` Here the intervals are chosen taking the class labels into account, so that they are useful for classification. Doing the discretization during the learning can be expensive for big data, so we use heuristic approaches with specific metrics to choose the cut points.

`ChiMerge.` It is a bottom-up method based on the χ^2 test. F is the numeric feature and K the class label.

1. We sort the records by F and we start from maximum detail: one interval for each distinct value of F, using as cut points the midpoints between consecutive values.
2. For each pair of adjacent intervals we build the contingency table (the two intervals on the rows, the classes on the columns) and we compute
$$\chi^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}, \qquad E_{ij} = \frac{(\text{total of row } i) \cdot (\text{total of column } j)}{N}$$

where O_{ij} are the observed counts, E_{ij} the counts expected under independence and N the number of samples in the two intervals. The null hypothesis is that the class is distributed in the same way in the two intervals: a small χ^2 means that there is no strong evidence that the two intervals differ, a large χ^2 that they behave differently.

3. We merge the pair of adjacent intervals with the smallest χ^2, if it is below the critical value. With two classes we have 1 degree of freedom, and with significance level $\alpha = 0.10$ the critical value is about 2.71.

4. We recompute χ^2 only for the new adjacent pairs and we repeat. We stop when no adjacent pair has χ^2 below the critical value.

ChiMerge is greedy, and it merges only adjacent intervals because the discretization must preserve the order of the values of F. The intervals become as coarse as possible while still preserving the changes that depend on the class: fewer intervals are simpler, but too much merging loses class information.

Example 3.11 (ChiMerge)**.**
We have 12 samples with $F = 1, 3, 7, 8, 9, 11, 23, 37, 39, 45, 46, 59$ and class $K = 1, 2, 1, 1, 1, 2, 2, 1, 2, 1, 1, 1$. The initial cut points are the midpoints $2, 5, 7.5, 8.5, 10, 17, 30, 38, 42, 45.5, 52$, that give 12 intervals between 0 and 60.
Two examples of the test. For the intervals containing the samples 2 and 3 (classes 2 and 1) every cell of the table has expected count 0.5, while the observed counts are 0 or 1: each of the four cells contributes $\frac{(0.5)^2}{0.5} = 0.5$, so $\chi^2 = 4 \cdot 0.5 = 2$. For the intervals containing the samples 3 and 4 (both of class 1) the expected counts are equal to the observed ones, so $\chi^2 = 0$.

- First pass: the adjacent intervals with the same class have $\chi^2 = 0$ and are merged first. We obtain $[0, 2)$, $[2, 5)$, $[5, 10)$, $[10, 30)$, $[30, 38)$, $[38, 42)$, $[42, 60]$, with χ^2 between neighbors equal to $2, 4, 5, 3, 2, 4$.
- The two pairs with $\chi^2 = 2 < 2.71$ are merged: $[0, 5)$, $[5, 10)$, $[10, 30)$, $[30, 42)$, $[42, 60]$, with χ^2 equal to $1.875, 5, 1.333, 1.875$.
- The smallest value is 1.333: we merge $[10, 30)$ and $[30, 42)$. Now we have $[0, 5)$, $[5, 10)$, $[10, 42)$, $[42, 60]$ with χ^2 equal to $1.875, 3.938, 3.938$.
- We merge $[0, 5)$ and $[5, 10)$. The remaining values are 2.723 and 3.938, both above 2.71, so we stop.

The final bins for F are $[0, 10)$, $[10, 42)$ and $[42, 60]$, that contain respectively 4, 1 and 3 samples of class 1 and 1, 3 and 0 samples of class 2.

`Fayyad and Irani approach.` It is a supervised, top-down method based on the entropy. For each continuous attribute A we look for the best set of cut points, evaluating the candidates with the class entropy.

We do not need to test every possible real value: the optimal cut points lie between two adjacent sorted examples of different classes. So we sort the examples by A and we consider only the boundary points.

Definition 3.12 (Boundary point)**.**
A potential cut point T is a boundary point if it lies between two adjacent examples e_1 and e_2 of different classes, with $A(e_1) < T < A(e_2)$. Usually T is the midpoint. B_A is the set of the candidate boundary points for the attribute A.

Definition 3.13 (Class entropy)**.**
Let S be a set of examples with k classes $C_1, \dots, C_k$ and let $P(C_i, S)$ be the proportion of examples of S that have class C_i. The class entropy of S is

$$E(S) = -\sum_{i=1}^{k} P(C_i, S) \log_2 P(C_i, S)$$

The entropy measures the uncertainty on the class: it is close to 0 when the subset is pure and it is high when the classes are mixed. A cut point T splits S into S_1 (the examples with $A \leq T$) and S_2 (the examples with $A > T$), and the class information entropy of the partition is the weighted average

$$EP(A, T; S) = \frac{|S_1|}{|S|} E(S_1) + \frac{|S_2|}{|S|} E(S_2)$$

The best cut point T_A is the candidate with the minimum $EP(A, T; S)$, the one that reduces the uncertainty the most.

The algorithm is recursive: we split S at the best cut point T_A, then we apply the same procedure to S_1 and S_2. We continue only while the split is worth its complexity. The information gain of the cut is

$$G(A, T_A; S) = E(S) - EP(A, T_A; S)$$

and, by the MDL (minimum description length) principle, the cut is accepted only if

$$G(A, T_A; S) > \frac{\log_2(|S| - 1) + \Delta(A, T_A; S)}{|S|}$$

otherwise we stop. $\Delta(A, T_A; S)$ is a correction term that penalizes the discretizations that are too complex:

$$\Delta(A, T_A; S) = \log_2(3^c - 2) - \left[c\, E(S) - c_1 E(S_1) - c_2 E(S_2)\right]$$

where c, c_1, c_2 are the numbers of classes represented in S, S_1, S_2.

`Concept hierarchy generation.` A concept hierarchy organizes the values of an attribute hierarchically, and we reduce the data by replacing the low-level values with higher-level concepts: for example the numeric ages become *youth*, *adult*, *senior*. In a data warehouse this allows us to view the data at multiple granularities (drill-down and roll-up). The hierarchy can be specified by the experts or formed automatically, both for numeric data (applying the discretization methods recursively) and for nominal data, for example street $<$ city $<$ state $<$ country, or {Urbana, Champaign, Chicago} $<$ Illinois. For nominal attributes the automatic generation can use the number of distinct values of each attribute.

4 Lesson 04 — 13/10/2026

4.1 Classification

We start to analyze the problem of `classification`. In classification problems we have a set of objects described by a set of attributes, the objective is to associate each object with a class based on the value of their attributes. (Ex. we can describe patients by age, family history, smoker and classify them in low risk and high risk). The first family of classifiers we will see are called `lazy learners`, a lazy learner does not build a model during the training phase. The classification is divided in two phases: `training`, we give the model labelled objects and it simply stores them, `prediction`, the model performs generalization on unlabelled objects (`test set`). The simplest lazy learner we can think of is the `Rote learner`: if we have an object identical to the tested one $\implies$ associate the same class; if it is not identical $\implies$ NO class.

This classifier works but can be improved for sure, because for many objects we don't have a class. This classifier is not able to generalize. To improve this classifier we need to remove the identical constraint and look for similar objects instead of a perfect match. For similarity we consider distance, this approach is called `1-Nearest neighbour`:

Algorithm 1: 1-Nearest Neighbour

Input: training set $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, new sample x
$best_distance \leftarrow +\infty$; $predicted_class \leftarrow$ null;
foreach $(x_i, y_i) \in D$ **do**
    $distance \leftarrow dist(x, x_i)$;
    if $distance < best_distance$ **then**
        $best_distance \leftarrow distance$; $predicted_class \leftarrow y_i$;
return $predicted_class$;

The complexity is $O(N \cdot F)$ where N is the number of objects and F the number of features, we need to check the distance with all the other objects to look for the nearest. This approach works well but there is a problem. If there is noise we can have errors and misclassification if the nearest neighbour is a wrongly classified object. We can solve this problem by increasing the number of neighbours, `K-Nearest Neighbour`, and choosing the majority class among them. This approach works and is robust to noise but we need to choose k and we don't have an algorithm to choose it, we need then to test and evaluate the accuracy to choose the best value of k. Another problem we have is that k-nearest neighbours is computationally heavy when the number of objects is very large.

By implementing the algorithm with a max-heap we can reduce the complexity to $O(NF + N \log K)$. The heap keeps the k closest points seen so far, its root is always the farthest among the current k neighbours.

Algorithm 2: Efficient k-Nearest Neighbour

Input: training set $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, query sample x, number of neighbours k
$H \leftarrow$ empty max-heap (at most k elements, keyed by distance);
foreach $(x_i, y_i) \in D$ **do**
    $d \leftarrow dist(x, x_i)$;
    if $size(H) < k$ **then**
        insert (d, y_i) into H;
    else if $d < maximum_distance(H)$ **then**
        remove the element with maximum distance from H;
        insert (d, y_i) into H;
$predicted_class \leftarrow$ majority vote of the labels in H;
return $predicted_class$;

We can improve the efficiency of lazy learners, the problem is the number of objects, if it is too large complexity increases. We need then to reduce the number of objects, without decreasing the accuracy of our classifier of course. We can look for zones where we have a lot of objects with the same class and replace them with a representative one. Another method is to implement `editing methods`: removal of training instances that do not provide useful information for the learning process.

4.1.1 Editing methods

`Wilson editing`: In Wilson's method we remove samples that disagree with the class vote of their neighbours.

Algorithm 3: Wilson editing

Input: training set O, number of neighbours K
Preprocessing;
foreach $(o_i, y_i) \in O$ **do**
    find the K nearest neighbours of o_i in O, excluding o_i;
    predict $\hat{y}_i$ as their most frequent class label (ties are broken randomly);
    mark o_i for removal if $\hat{y}_i \neq y_i$;
remove all marked points to obtain O_r;
Classification: classify each new example q using K-NN on the edited set O_r;

All the neighbourhoods use the original O, points are deleted only after the full pass. This approach does not remove a lot of instances but is interesting because it increases the accuracy of the classifier by removing instances considered noisy. Note that local disagreement does not necessarily mean that an example is mislabeled.

`Multi-edit`: a more robust iterative version of Wilson editing, the editing decision is less dependent on one single neighbourhood configuration. The steps are:

1. *Randomly split*: partition the current training set into M subsets.
2. *Classify*: for each subset S_i, use $S_{(i+1) \bmod M}$ as reference data for applying Wilson editing.
3. *Remove*: delete the instances that are misclassified.
4. *Merge*: join all the surviving instances into a new training set.
5. *Repeat*: stop when no more instances are removed.

Wilson editing evaluates each point using its neighbourhood in the whole reference set, multi-edit evaluates points through repeated random subset comparisons. A single edit may be sensitive to the chosen neighbourhood, the repetition exposes unstable points to different local contexts and removes the consistently suspicious ones.

`Citation editing`: To define the neighbours of an instance we use references and citers (as in a citation graph: if a paper cites another one the two are related, and also if it is cited by another one). For an instance x, the neighbours may include: 1. its nearest neighbours (references of x) and 2. the instances for which x is a neighbour (citers of x). Editing decision: the rule is evaluated on this richer relation, not only on a one-way k-NN list. We keep x only if its citation neighbourhood supports its label.

`Supervised clustering`: we replace the original training set D with a smaller set D_r of representative objects selected using class information. The steps are: run a supervised (class-aware) clustering on D, choose the representatives of the clusters, use the reduced set D_r as new training set. The edited training set stores a compact set of class-consistent prototypes. This can reduce storage and prediction time, while preserving the class structure needed by nearest-neighbour classifiers.

If we want to compare the editing methods we can see compression, accuracy, interpretation. The *compression rate* is the ratio between the size of the edited training set and the size of the original one. There is a trade-off: stronger compression can reduce noise, but excessive compression may remove useful boundary cases. The comparison below comes from the experimental study of Zeidat, Wang and Eick, "Dataset Editing Techniques: A Comparative Study".

Method	Compression	Accuracy	Interpretation
Wilson editing	moderate	often improves	removes local noise / overlap
Multi-edit	variable	more stable	repeated editing across subsets
Citation editing	method-dependent	context-aware	uses reverse-neighbour relations
Supervised clustering	high	depends on representatives	compresses data into prototypes

4.2 Prediction problems

Can I apply the k-NN to predict a number instead of a label? The answer is yes but we need to change a bit the algorithm. In the classification problem we choose the class with the majority vote, in numeric prediction we predict the value by averaging the values of the neighbours of the target point.

The k-NN regression algorithm works as follows:

1. *Store* the training data, with the examples together with their numeric target values y.
2. *Measure distances*: for a new sample x_q, compute the distance to each training example.
3. *Select* the k nearest examples, they define the local neighbourhood.
4. *Average* the target values of the neighbours.

The prediction is

$$\bar{y}(x_q) = \frac{1}{k} \sum_{i=1}^{k} y_i,$$

or, in the weighted option,

$$\bar{y}(x_q) = \frac{\sum_{i=1}^{k} w_i \, y_i}{\sum_{i=1}^{k} w_i}.$$

For example, with $k = 3$ and neighbours with targets 72, 75, 81 the prediction is $(72+75+81)/3 = 76$.

4.3 Case-Based Reasoning (CBR)

Another lazy learners algorithm is the CBR. If the training set is not a set of objects but a set of complex cases (or problems) we may want to classify them because we assume that similar problems have similar solutions. The problem we have is that we need to define a new similarity concept because distance now is meaningless. A CBR is applied using four steps: retrieve, reuse, revise and retain.

In a CBR cases are symbolic experiences and not just points in a Euclidean space, a CBR system needs to store cases in form of text containing a rich description: attributes, function graphs, contextual information, solution and outcome.

`Retrieve`. We need to define a domain suitable similarity and using that we select one or several cases. The goal is to retrieve a small set of highly relevant cases.

`Reuse`. We use the retrieved cases to propose a new solution to the new cases.

`Revise`. We need to test our solution and evaluate if we need to retain the solution or not using feedback (rating or correction).

`Retain`. We store the new solution to make the system learn from new experience.

Remark. CBR is useful when past experiences are a reliable source of knowledge. The quality of a CBR system depends heavily on the case representation, the similarity function and the retention policy

5 Lesson 05 — 15/10/2026

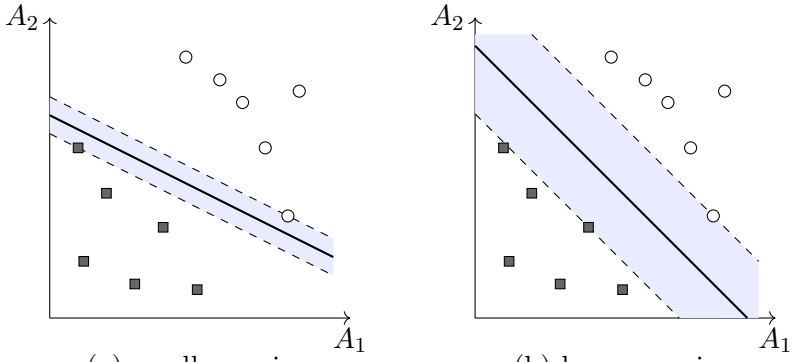
5.1 Eager learners

Differently from lazy learners, `eager` learners, when given a set of training data, construct a generalization model before receiving new data to classify. We need to build our own classifier and then evaluate whether it performs well or not. To evaluate its performance we need a ground truth that is similar to the training set in structure (it needs to have the class label), but different in values: the `test set`. To evaluate the classifier we need a metric. One option is the `accuracy`, which measures the percentage of test set instances that are correctly classified. This metric is not always a good choice when the classes are highly unbalanced.

Another frequent problem in classification is `overtraining` (overfitting). We have overtraining when the classifier fits the training data too closely, including noise and dataset-specific peculiarities, and therefore performs worse on unseen data. This can happen, for example, when the two classes are not well separated and overlap, possibly also because of noise. If there is overtraining, the classifier is too tuned on the training set and is not able to generalize well. To detect it we can first classify the instances in the training set, then the instances in the test set, and compare the two accuracies. If the test accuracy is much lower than the training accuracy, the classifier is likely overtrained; if the two accuracies are close, the classifier is generalizing well.

5.2 Linear support vector machine

A linear `support vector machine` (SVM) is a simple model that looks for a linear separator (a line) in the data. There could be an infinite number of lines that separate our data, but we want to find the "best" one, the one with the lowest classification error on unseen data. In higher dimensions we will use hyperplanes instead of lines, so from now on we use this term to refer to the decision boundary. In an SVM we look for the `maximum margin hyperplane` (MMH). By margin we mean the distance between the hyperplane and the closest sample of either class: the two *sides* of the margin are parallel to the hyperplane and at the same distance from it. Intuitively, a hyperplane with a larger margin should be more accurate on future data than one with a smaller margin.



(a) small margin (b) large margin

Both hyperplanes in the figure correctly classify all the training samples, but we prefer (b) because its margin is larger. To find the MMH we need to describe hyperplanes and margins with formulas. Let the training set be $(x_1, y_1), \ldots, (x_n, y_n)$, where $x_i \in \mathbb{R}^d$ is a sample and $y_i \in \{+1, -1\}$ is its class label. A separating hyperplane is a linear classifier and can be written as

$$H: \; w^T x + b = 0,$$

where $w = [w_1 \; \cdots \; w_d]^T$ is a weight vector and b is a scalar called *bias*. With two attributes this is just the line $b + w_1 x_1 + w_2 x_2 = 0$. Any point above the hyperplane satisfies $w^T x + b > 0$, and any point below it satisfies $w^T x + b < 0$. So the classifier is simply the sign of $w^T x + b$.

Now we describe the two sides of the margin. They are two hyperplanes parallel to H, so they have the form $w^T x + b = \pm c$ for some $c > 0$. Since w and b can be multiplied by the same positive constant without changing the separating hyperplane, we can choose a canonical scaling such that $c = 1$, so the two sides of the margin are

$$H_1 : \; w^T x + b = +1, \qquad H_2 : \; w^T x + b = -1.$$

Requiring that every sample lies on the correct side of its margin side gives

$$w^T x_i + b \geq +1 \quad \text{if } y_i = +1, \qquad w^T x_i + b \leq -1 \quad \text{if } y_i = -1,$$

which can be combined into a single constraint

$$y_i \left(w^T x_i + b\right) \geq 1 \quad \text{for all } i.$$

The samples lying on H_1 or H_2 satisfy the constraint with equality, $y_i \left(w^T x_i + b\right) = 1$: these are the `support vectors`. They are the samples closest to the separating hyperplane, the most difficult to classify, and they are the only ones that define the final classifier.

The margin can now be written only in terms of w. The distance of a point x from the hyperplane H is

$$\text{dist}(x, H) = \frac{|w^T x + b|}{\|w\|}.$$

For a support vector the numerator is 1, so each side of the margin is at distance $1/\|w\|$ from H. Hence the margin is

$$\rho = \frac{2}{\|w\|}.$$

Maximizing the margin is therefore equivalent to minimizing $\|w\|$. For mathematical convenience we minimize $\frac{1}{2}\|w\|^2$, which has the same minimizer, and we obtain the optimization problem

$$\min_{w,b} \; \frac{1}{2}\|w\|^2 \quad \text{subject to } y_i \left(w^T x_i + b\right) \geq 1, \quad i = 1, \ldots, n.$$

The constraints enforce correct separation, while the objective selects, among all feasible separators, the one with maximum margin.

We can solve it using Lagrange multipliers and the dual problem, which gives a maximization problem that reveals which samples become support vectors ($\alpha_i > 0 \iff x_i$ is a support vector). To predict the class of a new sample x we compute the inner products with the support vectors only:

$$f(x) = \text{sign}\Big(\sum_i \alpha_i y_i \, x_i^T x + b\Big).$$

If the result is positive, the sample is above the separator and is assigned to class $+1$, otherwise to class -1.

5.3 Nonlinear SVM and the kernel idea

The problem with SVM is that it is not always possible to linearly separate the classes, and in that case the optimization problem has no solution. To solve this problem we can apply a nonlinear mapping function $\Phi : x \to \varphi(x)$ to transform the data into a higher-dimensional space. This transformation is made easier by exploiting the `kernel function`. A kernel function is a function that is equivalent to an inner product in some feature space, $K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$: it implicitly maps data to a high-dimensional space.

Example 5.1.
Consider 2-dimensional vectors $x = [x_1\ x_2]^T$ and let $K(x_i, x_j) = (1 + x_i^T x_j)^2$. We need to show that $K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$:

$$\begin{aligned} K(x_i, x_j) &= (1 + x_i^T x_j)^2 \\ &= 1 + x_{i1}^2 x_{j1}^2 + 2\, x_{i1} x_{j1} x_{i2} x_{j2} + x_{i2}^2 x_{j2}^2 + 2\, x_{i1} x_{j1} + 2\, x_{i2} x_{j2} \\ &= \begin{bmatrix} 1 & x_{i1}^2 & \sqrt{2}\, x_{i1} x_{i2} & x_{i2}^2 & \sqrt{2}\, x_{i1} & \sqrt{2}\, x_{i2} \end{bmatrix} \begin{bmatrix} 1 & x_{j1}^2 & \sqrt{2}\, x_{j1} x_{j2} & x_{j2}^2 & \sqrt{2}\, x_{j1} & \sqrt{2}\, x_{j2} \end{bmatrix}^T \\ &= \varphi(x_i)^T \varphi(x_j), \end{aligned}$$

where $\varphi(x) = \begin{bmatrix} 1 & x_1^2 & \sqrt{2}\, x_1 x_2 & x_2^2 & \sqrt{2}\, x_1 & \sqrt{2}\, x_2 \end{bmatrix}^T$.

Some popular kernel functions are:

- `Linear`: $K(x_i, x_j) = x_i^T x_j$;
- `Polynomial` of power p: $K(x_i, x_j) = (1 + x_i^T x_j)^p$;
- `Gaussian` (radial-basis function): $K(x_i, x_j) = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}}$.

What happens if we have more than two classes? The standard SVM is naturally binary. To apply it to more than two classes we can follow the `one-vs-rest` strategy: we decompose the problem into identifying one single class against all the others, and we repeat this for every class. To classify new instances we choose the class with the highest score (we cannot use the sign as before, because we could have more than one positive result).

5.4 Decision tree

Another model we will see is the `decision tree`. This model is still widely used today because it is considered one of the most interpretable classifiers. This means that the classifier itself is able to explain how the given input is correlated to the output. Other very good characteristics of decision trees are: it requires no domain knowledge, it can be learned directly from the data, it can handle many attributes and there is no parameter to set. A decision tree is a flowchart-like structure where the internal `nodes` represent the attribute tests, the `branches` are the outcomes of the tests and in the `leaves` we find the class labels. The problem is: how can we generate the decision tree?

Of course the choice of which test to put in each node will affect the final result, so we need an algorithm to decide what to put in the nodes.

The main goal of the decision tree is to generate in the sample space some sub-spaces where there are only members of one class, so when I have to classify a new object that falls in one particular sub-space I can choose the class of that sub-space.

5.4.1 Algorithm for decision tree induction

Decision tree induction follows a greedy, top-down, recursive divide-and-conquer strategy:

- at each stage we make the locally optimal split;
- we hope that these local choices lead to a good global tree;
- the instances start at the root and are recursively partitioned.

We assume that the attributes are categorical: continuous attributes are discretized in advance. The test attribute is selected using a heuristic or statistical measure, for example the information

gain or related impurity measures. The recursion stops when all the instances at a node belong to the same class, when no attributes are left (and we use majority voting), or when no instances are left.

Algorithm 4: Decision tree induction

Input: a set D of training instances, a list of candidate attributes
Output: a decision tree
create a node N containing all the instances of D;
if *all the instances in D belong to the same class C* **then**
  return N as a leaf labelled with C;
if *the attribute list is empty* **then**
  return N as a leaf labelled with the majority class in D;
select the best test attribute A using the selection measure;
label N with A and remove A from the attribute list;
foreach *outcome a_j of A* **do**
  let D_j be the subset of instances in D with outcome a_j;
  if *D_j is empty* **then**
    attach to N a leaf labelled with the majority class in D;
  else
    attach to N the tree returned by the algorithm applied to D_j;
return N;

6 Lesson 06 — 20/10/2026

6.1 Some notation

From now on we will use D to denote the training set (the subset of training tuples reaching the current node) and $|D|$ for the cardinality of D (number of tuples). We consider m classes $C_1, \dots, C_m$ and we call p_i the empirical probability that a tuple in D belongs to class C_i:

$$p_i = \frac{|\{x \in D : y(x) = C_i\}|}{|D|}.$$

Then we can define the `entropy` as the expected information needed to classify a tuple in D. The lower the entropy, the purer D is (it is 0 when all the tuples belong to the same class).

Definition 6.1 (Entropy)**.**

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2 p_i.$$

A candidate attribute A with v different outcomes $a_1, \dots, a_v$ will partition D into v subsets $D_j = \{x \in D : A(x) = a_j\}$, $j = 1, \dots, v$. We can then measure the `weighted entropy` after splitting on A as the weighted average of the entropies of the child nodes:

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} Info(D_j).$$

When choosing a partition we need to be careful not to choose partitions that lead to a single instance in each node: the entropy will be 0 because every child is a pure partition, but the split is useless for classifying new data.

The `information gain` is the reduction in entropy after the split.

Definition 6.2 (Information gain)**.**

$$Gain(A) = Info(D) - Info_A(D).$$

We want to maximize $Gain(A)$, so at each node we select the attribute with the highest information gain (this is what ID3 does).

6.1.1 Split-point search

For a continuous attribute A we cannot create one branch per value, so we look for a threshold s and make a binary split ($v = 2$):

$$D_1(s) = \{x \in D : A(x) \leq s\}, \qquad D_2(s) = \{x \in D : A(x) > s\}.$$

We sort the observed values of A in increasing order, $a_1 \leq a_2 \leq \cdots \leq a_n$. The candidate thresholds are the midpoints between adjacent values,

$$s_\ell = \frac{a_\ell + a_{\ell+1}}{2}, \qquad \ell = 1, \ldots, n-1,$$

so there are at most $n - 1$ candidates. For each of them we compute the weighted entropy of the two children, and we keep the threshold that minimizes it (that is, the one with the highest gain):

$$s^* = \arg\min_s Info_{A,s}(D).$$

The node then tests $A \leq s^*$ versus $A > s^*$.

6.2 Decision tree induction: top-down greedy learning

At each node the algorithm chooses the best local split and recursively partitions the data (see the algorithm in the previous lesson):

1. all the training instances start at the root;
2. we select a test attribute using a heuristic or statistical measure (information gain, gain ratio, Gini index, χ^2, ...);
3. each child receives the subset D_j of instances satisfying its branch condition, and we repeat recursively;
4. we stop when all the instances have the same class, no attributes remain or no instances are left.

A partition is pure when all its tuples belong to the same class: good splits produce children that are purer than the parent node.

6.2.1 Gain ratio

Information gain can favor attributes with a lot of different outcomes, such as unique identifiers, that create many tiny, apparently pure partitions. C4.5, the successor of ID3, uses the `gain ratio` to prevent that: it normalizes the information gain by the `split information`, the information generated by the split itself.

Definition 6.3 (Gain ratio)**.**

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \log_2 \frac{|D_j|}{|D|}, \qquad GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}.$$

We then select the attribute with the maximum gain ratio.

Remark. The gain ratio can prefer highly unbalanced splits: when $SplitInfo_A(D)$ is close to 0 the ratio becomes very large.

6.3 Overfitting

If we have noise and irregularities in the training set and the classifier is too tuned on the training set, it will also be tuned on the noise: this results in `overfitting` and in a decrease of the generalization performance. The overfitted tree is often large, with too many branches. To avoid overfitting we need to reduce the complexity of the classifier; of course, if it is too simple we can have `underfitting` and not be able to classify well, so we need a compromise.

To detect overfitting we can follow the assessment procedure we already discussed in the previous lessons: compare the training accuracy with the accuracy on an independent test (or validation) set. The test set must not be used to build the tree, otherwise the estimate of the generalization performance is biased.

Some notation:

- `re-substitution error`: the error computed on the same training set used to learn the tree. It is often too optimistic, because the tree has been fitted to these examples;
- `generalization error`: the error expected on independent, previously unseen samples. It is the quantity we really want to minimize;
- `validation` (or `pruning`) `set`: a separate dataset used only to estimate whether pruning a subtree is beneficial.

For a tree T evaluated on n samples:

$$e_{TR}(T) = \frac{\#\text{training misclassifications}}{n_{TR}}, \qquad e_{TS}(T) = \frac{\#\text{test or validation misclassifications}}{n_{TS}}.$$

The optimistic estimate takes $e_{TS}(T) \approx e_{TR}(T)$. With reduced error pruning we use a pruning set to estimate $e_{TS}(T)$ directly, while the pessimistic estimate penalizes the training error by the number of leaves.

6.4 Pruning

We can use pruning to reduce the tree complexity and overfitting. There are two different strategies: `pre-pruning` and `post-pruning`.

- In pre-pruning we prune the tree during its construction: we stop splitting when some stopping criterion is met (for example maximum depth, minimum number of samples per node, minimum impurity decrease), so we need to fix some thresholds. The main problem is that we could stop too early and underfit.
- In post-pruning we prune after growing a large tree, by replacing subtrees with leaves using an estimated-error criterion. The new leaf is labelled with the majority class among the examples reaching that node. The main limitation is that it requires growing and evaluating a large tree.

Usually post-pruning is safer than pre-pruning, because potentially useful splits are not rejected too early.

6.4.1 Reduced error pruning

In `reduced error pruning` (REP) we use a separate pruning set to evaluate whether a subtree needs to be removed or not. Let T_v be the subtree rooted at node v; on the pruning set we compute

- $misclass(T_v)$: number of errors made by the subtree;
- $misclass(v)$: number of errors if v is replaced by a leaf;

and then

$$Gain_prune(v) = misclass(T_v) - misclass(v).$$

If $Gain_prune(v) > 0$, pruning reduces the validation error. If it is zero, the simpler tree is often preferred. Starting from the fully grown tree, we evaluate the possible pruning operations bottom-up, prune the node with the largest positive gain and repeat until all the remaining gains are negative.

6.4.2 Interpretability

Generally pruning improves the tree but does not solve everything. Some structural problems may remain:

- `repetition`: the same attribute is tested multiple times along a single root-to-leaf path, which makes the path longer and less readable;
- `replication`: similar or identical subtrees appear in different parts of the tree, which increases its size without adding much explanatory value.

As alternatives, we can use multivariate splits, or extract IF-THEN rules from the tree and prune or simplify the rules directly.

Decision trees are considered very interpretable classifiers, because we can transform the tree into rules: each root-to-leaf path becomes an IF-THEN rule. We can have `global interpretability`, related to the overall structure of the tree (we want it not too big), and `local interpretability`, where for a specific instance we can follow its unique path in the tree to understand the prediction.

6.5 Bayesian classifier

A Bayesian classifier is a statistical classifier: instead of returning only a label, it returns for every class the probability that the instance belongs to that class. As classification criterion we choose the class with the highest probability. One problem is that we have to compute many probabilities, and this could be heavy. We will see the `naive Bayes classifier`.

Notation (slide 89):

Symbol	Meaning
X	observed data tuple (the evidence), class label unknown
H	hypothesis: X belongs to a class C
$P(H)$	prior probability of the hypothesis
$P(X)$	evidence: probability that X is observed
$P(X \mid H)$	likelihood: probability of observing X if H holds
$P(H \mid X)$	posterior: probability that H holds after observing X

By Bayes' theorem $P(C_i \mid X) = P(X \mid C_i)\, P(C_i)/P(X)$, and we predict the class C_i with the largest posterior. Since $P(X)$ is the same for all the classes, it is enough to maximize $P(X \mid C_i)\, P(C_i)$.

To reduce the number of probabilities to compute we use the `naive assumption`: given the class C_i, the attributes are conditionally independent, so

$$P(X \mid C_i) = \prod_{k=1}^{n} P(x_k \mid C_i).$$

The assumption is useful even when it is not perfectly true: it turns a hard joint-probability estimation problem into several reliable class-conditioned frequency estimates.

The naive Bayes classifier can have a problem: a single zero conditional probability makes the whole product zero. To solve this we use the `Laplacian correction`: we add one pseudo-count to each possible value, so that no probability is zero.

Advantages:

- easy to implement;
- often gives good results;
- computationally efficient, because it mainly requires class-conditional counts.

Limitations:

- it assumes class-conditional independence, while real variables are often dependent.

6.6 Bayesian belief networks

When the independence assumption does not hold we need richer models, such as `Bayesian belief networks`. They are defined by a directed acyclic graph and by `conditional probability tables` (CPT): nodes are random variables and links represent dependencies. The graph encodes conditional independencies that simplify the joint distribution: each variable is conditionally independent of its non-descendants given its parents, so

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid Parents(X_i)).$$

For classification, any node can be selected as the output node, representing the class label (there may also be more than one output node), and the classifier can return a probability distribution over the classes.

7 Lesson 07 — 22/10/2026

7.1 Rule-based Classification

The idea is to use a set of rules to determine the class of an entity. We can use IF-THEN rules, for example

$$\text{IF } age = \text{youth AND } student = \text{yes THEN } buys_computer = \text{yes}.$$

How can we measure if a rule is good? We can use `accuracy` to measure how many of the tuples that activate the rule are correctly classified. Another metric is the `coverage`: it measures how much of the dataset is covered by a rule. If a rule applies to only one tuple it is not a good rule (but it will have 100% accuracy).

Definition 7.1 (Coverage and accuracy)**.**
Let R be a rule, D the dataset, n_{covers} the number of tuples covered by R and $n_{correct}$ the

number of those tuples that R classifies correctly. Then

$$coverage(R) = \frac{n_{covers}}{|D|}, \qquad accuracy(R) = \frac{n_{correct}}{n_{covers}}.$$

A problem with rule-based classification is that we could have a conflict if more than one rule fires for an instance. To deal with conflicts we can use priorities:

- `Size ordering`: we give the highest priority to the rules with the toughest requirements, i.e. the most attribute tests in the antecedent.
- `Class-based ordering`: we order the classes by prevalence (rules for the most frequent class come first) or by misclassification cost (rules for the class with the highest cost come first).
- `Rule-based ordering`: we put the rules in a list ordered by rule quality (accuracy, coverage, size or domain advice) and apply them in order, like an access control list. This list is called a *decision list*.

7.1.1 Rules from decision trees

We can easily extract rules from a decision tree: each path from the root to a leaf becomes one rule.

Example 7.2.
Rules extracted from the *buys_ computer* decision tree (slide 112).

Path	Extracted rule
age = youth → student = no	IF age = youth AND student = no THEN class = no
age = youth → student = yes	IF age = youth AND student = yes THEN class = yes
age = middle_aged	IF age = middle_aged THEN class = yes
age = senior → credit_rating = fair	IF age = senior AND credit_rating = fair THEN class = yes
age = senior → credit_rating = excellent	IF age = senior AND credit_rating = excellent THEN class = no

Rules from a tree are mutually exclusive and exhaustive. However, repetition and replication in the tree can produce a large rule base that is hard to interpret. To avoid that we can apply pruning to the rules. Each condition that does not improve the estimated accuracy of the rule can be pruned, and any rule that does not contribute to the overall accuracy can be removed. C4.5 uses a pessimistic estimate, because the accuracy is evaluated on the training set and would otherwise be too optimistic.

After pruning, the rules may no longer be mutually exclusive and exhaustive. So a `default class` is used for the tuples not covered by any rule. To handle conflicts, C4.5 groups the rules by class and ranks the class rule sets to minimize false positives: the class rule set with the fewest false positives is examined first.

7.1.2 Sequential covering

Instead of building a tree we can directly generate rules from the training data, using the `sequential covering method`. The core idea is that sequential covering learns one rule at a time for a target class C_i. A rule should cover many positive examples of C_i and none, or only a few, examples of other classes. After learning one rule we remove the positive instances covered by the rule, find another rule with the same characteristics and repeat. We stop when some stopping condition holds.

But how is it possible to learn one rule? We start from the most general rule we can think of: IF true THEN C_i. We start from a condition that is always true and selects the whole training set (not a good rule in terms of accuracy). Then we add more and more predicates to the rule: at each step we evaluate all the possible predicates and choose the one that improves the rule-quality measure the most (so we need to define rule quality).

We can evaluate the quality of a rule, as we said, with the `accuracy`. Let p and n be the numbers of positive and negative examples covered by R:

$$accuracy(R) = \frac{p}{p+n}.$$

With accuracy alone we can be misled when very few instances are counted, so we need to consider also the `coverage`. Let P be the set of positive examples (the examples of the target class):

$$coverage(R) = \frac{p}{|P|}.$$

A useful rule should be both accurate and have a high coverage to be considered a good rule.

For example, the algorithms FOIL and RIPPER use this formula to decide which predicate is the best:

$$\text{FOIL-gain}(p) = pos' \left(\log_2 \frac{pos'}{pos' + neg'} - \log_2 \frac{pos}{pos + neg} \right),$$

where pos and neg are the numbers of positive and negative examples covered by the current rule R, and pos' and neg' are the ones covered by the rule R' obtained by adding the predicate p to R. A predicate is useful if it increases the rule purity while keeping enough positive examples covered.

One problem with Learn-One-Rule is that if rules are too specific there could be overfitting, and the classifier will not be able to generalize well. The solution is pruning: remove one conjunct at a time and check whether the pruned rule performs better on an independent pruning set.

7.2 Model evaluation and selection

We have now studied a good number of different classifiers and models. How can we evaluate which is the best for our case study?

We need some metrics to evaluate performance. The most intuitive one is the accuracy, and we have already discussed it. Accuracy could be misleading, for example, in the case of imbalanced classes.

We need more detailed information to evaluate how the classifier is working. We can use the `confusion matrix`, a compact summary of how actual classes are mapped to predicted classes. The cell CM_{ij} contains the number of instances of class C_i classified as C_j. On the diagonal we have the correctly classified instances.

Example 7.3.
Confusion matrix for two classes C and $\neg C$ (slide 169). Rows are the actual classes, columns the predicted ones.

Actual \ Predicted	C	$\neg C$	Total
C	TP (true positives)	FN (false negatives)	P
$\neg C$	FP (false positives)	TN (true negatives)	N
Total	P'	N'	All

TP and TN are correct decisions, while FP and FN are two different kinds of errors.

This matrix gives us all the information we need to evaluate a classifier. However, comparing two different classifiers through their whole matrices may not be easy, so we can instead use the following pairs of metrics, derived from the matrix.

The first pair is:

- `Sensitivity`: $\frac{TP}{P} = \frac{TP}{TP+FN}$;
- `Specificity`: $\frac{TN}{N} = \frac{TN}{TN+FP}$.

Sensitivity and specificity should be used together. Sensitivity measures how well the classifier recognizes the positive tuples, while specificity measures how well it recognizes the negative ones (slide 170). Each one alone can be misleading: a classifier that labels every tuple as positive has sensitivity 1 but specificity 0. This is important with imbalanced classes, where a high accuracy may simply reflect the majority class.

Example 7.4.
Cancer screening on 10000 tuples (slide 173): $P = 300$ patients have cancer and $N = 9700$ do not. The classifier gets $TP = 90$ and $TN = 9560$, so

$$sensitivity = \frac{90}{300} = 30.00\%, \qquad specificity = \frac{9560}{9700} = 98.56\%,$$
$$accuracy = \frac{90 + 9560}{10000} = 96.50\%.$$

The accuracy is high, but the classifier detects only 30% of the cancer cases. Looking at sensitivity and specificity together reveals the problem.

Another pair is `precision` and `recall`, and they represent the exactness and the completeness of a classifier. They need to be used together as well, because a perfect precision score (no false positives) does not reveal how many positives were missed, and a perfect recall score does not reveal how many negatives were incorrectly included.

$$precision = \frac{TP}{TP+FP}, \qquad recall = \frac{TP}{TP+FN}.$$

What happens in the case of a multi-class classifier? We compute the evaluations using a `one-vs-rest` approach: we treat the class C_i as positive and all the other classes as negative, and we then compute the confusion matrix (TP_i, FP_i, TN_i, FN_i) for each class. From these we get the per-class metrics ($precision_i$, $recall_i$, ...), which can be combined into a single classifier-level metric (slide 174):

- `Macro average`: simple average across the classes;
- `Weighted average`: average weighted by the class size;
- `Micro average`: the counts TP_i, FP_i, TN_i, FN_i are aggregated over all classes before computing the metric.

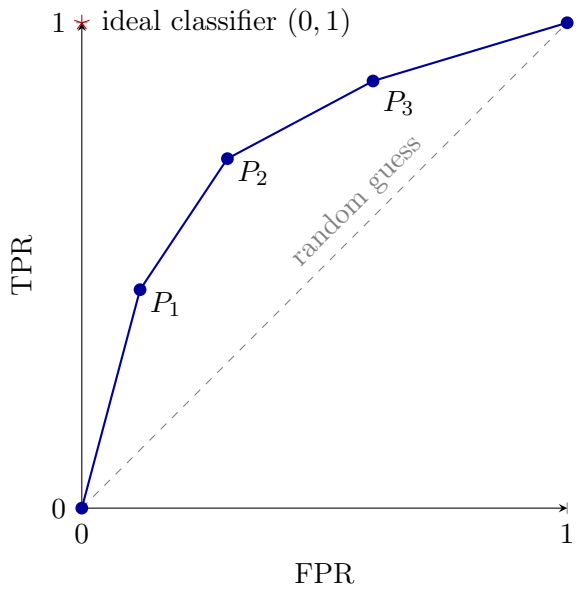
7.2.1 Area under curve

Let's take all the steps to define the area under curve metric.

First we need to define the `ROC curve` (Receiver Operating Characteristic), plotting

$$TPR = \frac{TP}{P} \quad \text{and} \quad FPR = \frac{FP}{N}.$$

Note that TPR is the sensitivity and $FPR = 1 - specificity$. This curve shows the trade-off between the true positive rate and the false positive rate as the decision threshold varies. The ideal classifier should have $TPR = 1$ and $FPR = 0$. The ROC curve is obtained by changing the parameters of the classifier and plotting one point (FPR, TPR) for each setting.



To evaluate if the classifier is good we need to evaluate how close we are to the ideal case $(0, 1)$. To decide which classifier is closer to the ideal case we can evaluate the `area under curve` (AUC): the higher the area, the closer the classifier is to the ideal one. A perfect classifier has $AUC = 1$, while a classifier close to the diagonal (random guessing) has AUC close to 0.5.

8 Lesson 08 — 03/11/2026

8.1 Cost of a classifier

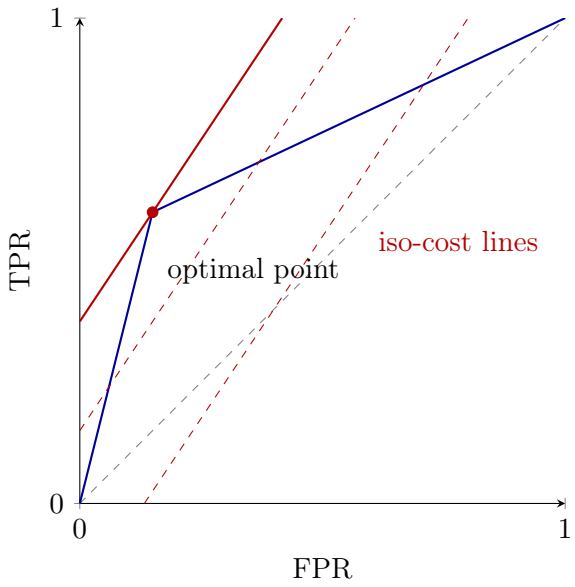
We want to evaluate the cost of a misclassification for both positive and negative instances. A classifier is a point (FPR, TPR) on the ROC plane, and its cost is

$$cost = FPR \cdot P(n) \cdot c(Y, n) + FNR \cdot P(p) \cdot c(N, p), \qquad FNR = \frac{FN}{P} = 1 - TPR.$$

$P(n)$ and $P(p)$ are the a-priori probabilities of a negative and of a positive example. $c(Y, n)$ and $c(N, p)$ are the false-positive and false-negative costs. Once probabilities and costs are fixed, the points with the same cost lie on a line of slope

$$slope = \frac{P(n)\, c(Y, n)}{P(p)\, c(N, p)},$$

so we obtain a family of parallel `iso-cost lines`. On the ROC plane the cost function is therefore a line, and we want to find the point of the ROC curve that is tangent to the lowest-cost line: this point is the optimal threshold for our classifier.



8.2 Estimating Classifier Accuracy

To decide which classifier is the best for our application domain we can't base our evaluation on only one trial, also because we can have different results depending on the training and test sets (we have only one labelled dataset). The solution is to repeat the evaluation under different partitions of training and test sets, and then average the results. After this process we will have a statistical distribution of results. How can we compare distributions? We can use the mean and the standard deviation of the results, but we need to be careful because the sample mean and the sample standard deviation are also random variables, so we need to estimate a confidence interval (this is what Stea said and I think is better). For prof. Marcelloni we need to perform some statistical tests to compare the distributions, for example the t-test.

Some simple methods based on repeated train/test partitions are the `holdout method` and `random subsampling`.

Algorithm 5: Holdout method

Input: dataset D, training fraction (e.g. 2/3)
Output: accuracy estimate
Randomly partition D into two independent sets: a training set D_{train} (e.g. 2/3 of D) and a test set D_{test} (e.g. 1/3 of D);
Build the model M on D_{train};
Estimate the accuracy of M on D_{test};

Algorithm 6: Random subsampling

Input: dataset D, number of repetitions k
Output: accuracy estimate
for $i = 1$ **to** k **do**
    Apply the holdout method on a new random partition of D, obtaining the accuracy acc_i;
return $\frac{1}{k} \sum_{i=1}^{k} acc_i$;

`k-fold Cross-validation`: We start by randomly partitioning the dataset into k mutually exclusive subsets of approximately equal size (`folds`) $D_1, \ldots, D_k$. At each iteration i we use the fold D_i as test set and the remaining $k-1$ folds as the training set. The accuracy is the average of the k estimates. Usually k equals 10 because with 10 iterations we can have some statistical significance.

8.2.1 Null hypothesis and paired comparison

To compare two classifiers M_1 and M_2 we use paired samples: both classifiers are evaluated on the same k folds. At each fold i we compute $err(M_1)_i$ and $err(M_2)_i$ and we analyze the paired differences

$$d_i = err(M_1)_i - err(M_2)_i.$$

Pairing reduces the variability due to different partitions. The `null hypothesis` is that the two classifiers have the same error on average, i.e. the expected difference is zero:

$$H_0 : \mu_d = 0.$$

The test assumes that the differences are approximately normally distributed and that the variances of the two populations are not reliably different. If H_0 is rejected, the observed difference is statistically significant and we choose the model with the lower estimated error rate.

8.3 Paired t-test

The paired t-test is a formal statistical test for the mean difference in cross-validation errors.

Algorithm 7: Paired t-test

Input: errors $err(M_1)_i$, $err(M_2)_i$ for $i = 1, \dots, k$ on the same folds; significance level α
Output: reject H_0 or not
Compute the paired differences $d_i = err(M_1)_i - err(M_2)_i$, $i = 1, \dots, k$;
Estimate the mean difference $\bar{d} = \frac{1}{k} \sum_{i=1}^{k} d_i$;
Compute the sample variance of the differences $s_d^2 = \frac{1}{k-1} \sum_{i=1}^{k} (d_i - \bar{d})^2$;
Compute the statistic $t = \dfrac{\bar{d}}{s_d/\sqrt{k}}$;
Set the degrees of freedom $df = k - 1$, which determine the shape of the t-distribution;
if $|t| > t_{1-\alpha/2,\,k-1}$ **then**
  reject H_0 (the mean difference is statistically different from zero)
else
  do not reject H_0 (the observed difference may be due to chance)

For example, for $k = 10$ and $\alpha = 0.05$ the critical value is $t_{0.975,\,9} \approx 2.262$.

Remark. In k-fold cross-validation the k estimates are not fully independent, because the training sets overlap, so the simple paired t-test can underestimate the variance. For a more rigorous comparison one can use the corrected resampled t-test, the 5×2cv test, or repeated/nested cross-validation protocols. Moreover, failing to reject H_0 does not mean that the two classifiers are equivalent: it only means that there is insufficient statistical evidence to conclude that their expected performances differ.

9 Lesson 09 — 05/11/2026

9.1 Wilcoxon signed-rank test

`Wilcoxon signed-rank test` is one of the most common `nonparametric tests`. The core idea is to compare paired measurements by ranking the absolute differences between the pairs and then restoring their signs. The test is used when the assumptions of the paired t-test are not met, particularly when the data is not normally distributed. Some cases of use are:

- the same units are measured twice, or observations are naturally matched;
- the differences are ordinal or continuous and a normal model is not appropriate.

The null hypothesis is that the median of the differences between the pairs is zero (H_0 : median$(d_i) = 0$). Under H_0, positive and negative signs should be distributed approximately at random across the ranks.

Algorithm 8: Wilcoxon signed-rank test (statistic)

Input: paired observations (x_i, y_i), $i = 1, \ldots, n$
Output: rank sums W^+ and W^-
Differences: compute $d_i = x_i - y_i$ for each pair and remove the pairs with $d_i = 0$;
Ranks: rank $|d_i|$ from the smallest to the largest, assigning the average rank to ties;
Signs: restore the sign of each original difference to its rank;
Rank sums: compute W^+ as the sum of the positive ranks and W^- as the sum of the negative ranks;

Here n is the number of non-zero paired differences, and

$$W^+ + W^- = \frac{n(n+1)}{2}.$$

9.1.1 Statistic and p-value

Under H_0 the observed statistic is

$$W = \min(W^+, W^-).$$

`Exact inference` is used for small samples: use the exact distribution of W or a critical-value table, find the probability of observing W or a more extreme value, and choose a one-sided or two-sided alternative before looking at the result. The p-value is compared with the significance level α; if a table gives only one-sided probabilities, a two-sided value is obtained with the convention stated by the table.

For larger samples the `normal approximation` is used. With the rule adopted in the course, it applies when $n(n+1)/2 > 20$. Under H_0,

$$\mu_W = \frac{n(n+1)}{4}, \qquad \sigma_W^2 = \frac{n(n+1)(2n+1)}{24}, \qquad z = \frac{W - \mu_W}{\sigma_W}.$$

The z-score states how many standard deviations the observed W lies below or above its mean under H_0. Pairs with $d_i = 0$ are removed and n is reduced accordingly.

Tied absolute differences. Assign the average of the tied ranks to every tied observation. For each group of t tied ranks, reduce the variance by $(t^3 - t)/48$. The correction affects the variance, and therefore the z-score, but not the observed rank sums.

Example 9.1.
Two classifiers M_1 and M_2 are evaluated on the same 10 folds, with $d_i = M_2 - M_1$.

Fold	1	2	3	4	5	6	7	8	9	10
M_1	2.0	3.6	2.6	2.6	7.3	3.4	14.9	6.6	2.3	2.0
M_2	3.5	5.7	2.9	2.4	9.9	3.3	16.7	6.0	3.8	4.0
d_i	+1.5	+2.1	+0.3	−0.2	+2.6	−0.1	+1.8	−0.6	+1.5	+2.0

Ranking $|d_i|$ and restoring the signs:

$\lvert d_i \rvert$	0.1	0.2	0.3	0.6	1.5	1.5	1.8	2.0	2.1	2.6
Rank	1	2	3	4	5.5	5.5	7	8	9	10
Sign	−	−	+	−	+	+	+	+	+	+

The signs are attached to the ranks of the absolute differences, not to the original values.

The rank sums are

$$W^- = 1 + 2 + 4 = 7, \qquad W^+ = 3 + 5.5 + 5.5 + 7 + 8 + 9 + 10 = 48,$$

and indeed $W^+ + W^- = 55 = 10 \cdot 11/2$. Hence $W = \min(W^+, W^-) = 7$.
Since $n(n+1)/2 = 55 > 20$ we use the normal approximation. There is one group of two tied ranks (the two 1.5), so the variance is reduced by $(2^3 - 2)/48 = 0.125$:

$$\mu_W = \frac{10 \cdot 11}{4} = 27.5, \qquad \sigma_W^2 = 96.25 - 0.125 = 96.125, \qquad z = \frac{7 - 27.5}{\sqrt{96.125}} = -2.091.$$

The two-sided p-value for $|z| = 2.091$ is $p \approx 0.037$. At $\alpha = 0.05$ we reject H_0: the results of the two classifiers differ significantly.

Remark. The p-value indicates evidence of a difference, not its practical magnitude: report also an effect estimate (e.g. the mean or median difference) and its uncertainty.

9.2 Neural Networks

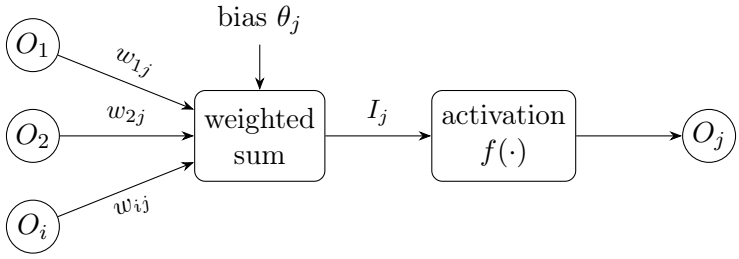
9.2.1 Multilayer feedforward networks

This is an old but classic architecture. We can identify three types of layer: the input layer, one or more hidden layers and the output layer. The input and output layers have a fixed number of neurons, so our parameters are the number of hidden layers and the number of neurons in each of them. A characteristic of this architecture is that it is `fully connected`: every neuron sends its output to each neuron in the next forward layer. The input layer is not counted as a computational layer, since it only passes the values forward; a "two-layer network" is therefore a network with one hidden layer and one output layer.

Inside a neuron. A neuron is made of two parts: a weighted sum of the inputs plus a bias, followed by an activation function f:

$$I_j = \sum_i w_{ij} O_i + \theta_j, \qquad O_j = f(I_j),$$

where w_{ij} is the weight of the connection from unit i of the previous layer to unit j, O_i is the output of unit i and θ_j is the bias of unit j.



The activation function should be nonlinear, because if it were linear, stacking layers would still behave like a linear model. A typical choice is the `logistic sigmoid`

$$O_j = \frac{1}{1 + e^{-I_j}},$$

which squashes the output between 0 and 1, useful for probabilities or binary outputs.

Learning. The parameters to be learned are the weights and the bias of every neuron. Learning is usually long and requires several attempts, because we also need to find the best network structure: basically, we train different neural networks and keep the best one. The parameters need to be initialized, and some initializations may lead to a local optimum instead of the global one, so we need to perform more than one random initialization.

Input representation. Numeric inputs are typically normalized, for example to $[0, 1]$. Discrete and categorical attributes are encoded, usually with `one-hot encoding`: there is one input neuron for each attribute value and only one is active at a time. For example, if A has values $\{a_0, a_1, a_2\}$ we use input units I_0, I_1, I_2, and the observed value a_1 is encoded as $(0, 1, 0)$. This avoids imposing an artificial numerical ordering among the categories.

Classification. If we want to use a neural network as a classifier, we put one neuron per class in the output layer; the predicted class is the one whose neuron gives the highest output.

9.2.2 Backpropagation

One of the most popular approaches to training is `backpropagation`: the output predicted by the model is compared with the actual output, the difference is propagated backward through the network, and the weights are modified in order to reduce this difference.

9.3 Ensemble Methods

The core idea is to combine multiple learned models to obtain a more reliable one: we learn k models $M_1, \ldots, M_k$ and combine their predictions into an improved model M^*, using the diversity among the models to reduce errors. There are three popular strategies:

- `Bagging`: average or vote over bootstrap classifiers;
- `Boosting`: weighted vote of sequential classifiers;
- `Heterogeneous ensembles`: combine different model families.

Example 9.2.
Why do ensembles work? Consider 25 independent binary classifiers, each with error probability 0.35. The ensemble, using majority voting, is wrong only if 13 or more classifiers are wrong. Let $X \sim \text{Binomial}(25, 0.35)$ be the number of wrong classifiers:

$$P(\text{ensemble error}) = P(X \geq 13) = \sum_{i=13}^{25} \binom{25}{i} 0.35^i \, 0.65^{25-i} \approx 0.060,$$

against 0.35 of a single classifier. The key assumption is independence: the reduction is large only if the classifiers make different mistakes, and correlated errors reduce the benefit of the ensemble.

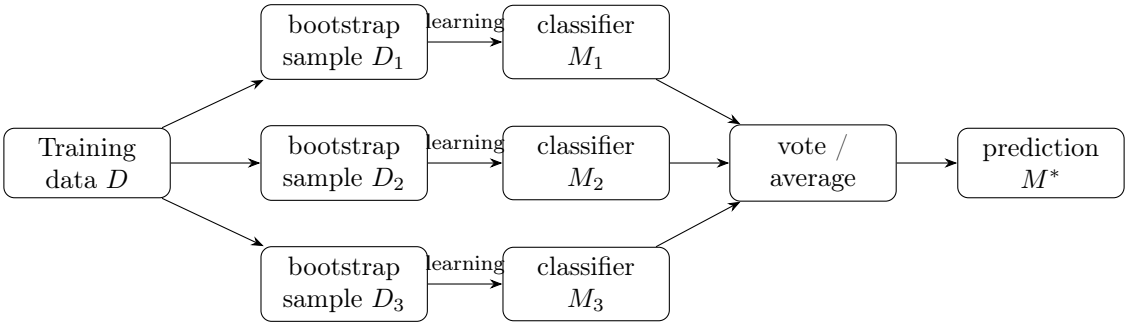
9.3.1 Bagging

Bagging (`bootstrap aggregation`) trains many models on bootstrap samples and aggregates their predictions. A `bootstrap sample` of a dataset D with d tuples is obtained by sampling d tuples from D *with replacement*: some tuples appear several times and others do not appear at all. Bootstrap samples therefore create diverse learners, whose predictions are then combined. The steps are:

1. create the bootstrap samples: from D with d tuples, sample d tuples with replacement to create each D_i;

2. train one model per sample: learn a classifier M_i from each bootstrap training set D_i, always with the same learning algorithm;

3. aggregate the predictions: for classification, count the votes and assign the class with the most votes.



Bagging is useful for both classification and numeric prediction. For the prediction of continuous values we apply all the models to a test tuple and average the predicted values, obtaining a more stable numeric estimate; the variance is reduced through aggregation.

Accuracy and robustness:

- it is often better than a single classifier learned from D;
- it is not considerably worse with noisy data;
- improvements in prediction accuracy have been proven.

10 Lesson 10 — 10/11/2026

10.1 Boosting

In boosting we don't work in parallel: we learn one classifier at each step, and at each iteration we train a classifier that focuses on the instances misclassified by the previous one. In practice, we do that by assigning a weight to each instance: a high weight means that the instance was misclassified by the previous classifier. For the first classifier the weights are all the same, of course. The final ensemble model combines all the classifiers using a weight per classifier (the higher the weight, the higher the accuracy) and performs a weighted vote.

As an example we will see the algorithm `AdaBoost`, which generates k classifiers in k rounds using weighted training tuples. For each classifier the error is the sum of the weights of the misclassified tuples, so mistakes on high-weight tuples count more than mistakes on low-weight tuples:

$$error(M_i) = \sum_{j=1}^{|D|} w_j \cdot err(X_j),$$

where $err(X_j) = 1$ if tuple j is misclassified, 0 otherwise. The vote weight of the classifier is

$$\alpha_i = \log \frac{1 - error(M_i)}{error(M_i)},$$

so the lower the error, the higher the vote weight.

The algorithm, where k is a user-defined number of boosting rounds, works as follows:

1. *Initialize*: set $w_j = 1/|D|$ for all the training tuples.
2. *Sample and train*: draw a weighted sample D_i and train M_i.
3. *Measure the error*: compute $error(M_i)$ on the weighted training tuples.
4. *Update the weights*: increase the relative weight of the misclassified tuples (in practice, the weights of the correctly classified tuples are multiplied by $error(M_i)/(1 - error(M_i))$) and normalize. Repeat for k rounds.
5. *Combine*: the final prediction is the class with the largest sum of α_i-weighted votes.

Remark. If $error(M_i) > 0.5$ the classifier is worse than random: it is discarded and the round is repeated.

Bagging and boosting both aggregate many models, but they generate diversity in different ways.

	Bagging	Boosting
Training sets	resampled datasets	reweighted datasets
Model building	models built in parallel	models built sequentially
Variance / bias	reduces variance	reduces bias and can reduce variance
Risk	generally robust to noise	can overfit noisy or mislabeled cases

10.2 Random forest

This is one of the most popular ensemble models: the base classifier is a decision tree and we use an ensemble of them. But if the algorithm that creates a decision tree is deterministic, how can we create different decision trees starting from the same data? We need to introduce some difference between the trees. We can either use randomized subsampling (and use the rest of the data to evaluate the tree error), or we can randomly extract a subset of features and use only them to create the tree. For the random forest algorithm pruning is not included: we always grow the trees fully, and pruning is not necessary because the trees compensate each other for overfitting.

The classical algorithm is:

1. *Bootstrap sample*: randomly extract $|D|$ training instances with replacement; the instances not extracted are used to estimate the tree error.
2. *Random feature subset*: at each node, randomly choose f attributes out of the F available and compute the best split among them only.
3. *Grow the tree*: each tree is fully grown and not pruned.

In classification, if we need to predict a class we activate all the decision trees and then choose the majority class.

Advantages of random forests:

- often very accurate across many data sets;
- efficient on large databases;
- handle thousands of input variables;
- estimate variable importance;
- provide internal error estimates;

- can estimate missing data.

Limitations:

- they may overfit on some noisy classification or regression datasets;
- when categorical features have different numbers of levels, they can favor the features with more levels, so the feature-importance scores are not always reliable and their interpretation requires caution.

10.3 Imbalanced datasets

We already saw the problem of imbalanced datasets when talking about the evaluation of models. We now ask ourselves the following question: does an imbalanced dataset also affect the training phase?

The brief answer is yes, because an imbalanced dataset creates some bias in the models. To solve that we can follow different strategies, such as:

- `Oversampling`: resample the positive (rare) class, so that it weighs more in the training set.
- `Undersampling`: reduce the negative (numerous) class by removing some of its tuples.
- `Threshold-moving`: adjust the decision threshold t of the classifier, so that it is easier for a tuple to be assigned to the rare class.
- `Ensemble techniques`: combine multiple classifiers.

Multiclass imbalance remains difficult.

One of the most used algorithms in the literature is `SMOTE` (synthetic minority over-sampling): the core idea is to oversample the dataset using synthetic samples of the minority class, instead of simply duplicating existing ones. Synthetic samples are placed in the local neighborhood of the already existing minority instances. SMOTE does not create information from scratch: it interpolates between minority samples, so preprocessing and validation must be designed carefully. In particular, synthetic samples must be generated only inside each training fold, not before cross-validation, to avoid data leakage.

10.3.1 How to choose neighbours

1. For each minority sample, find its k nearest neighbours among the minority class samples.
2. Depending on the desired amount of oversampling, randomly select one or more of them.

For example, with 200% oversampling and $k = 5$, we choose two of the five nearest minority neighbours and generate one synthetic point in the direction of each selected neighbour.

10.3.2 Generating one synthetic sample

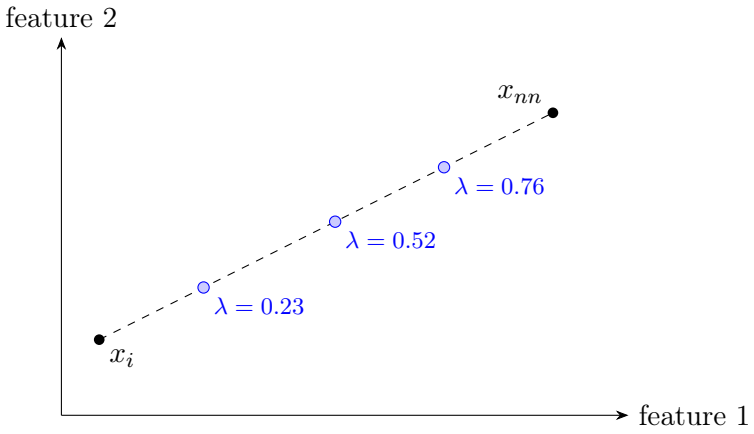
SMOTE interpolates between a minority sample and one of its minority neighbours:

1. take the difference between the neighbour vector and the current minority sample;
2. multiply it by a random value between 0 and 1;
3. add the result to the current sample.

Formally,

$$x_{new} = x_i + \lambda\,(x_{nn} - x_i),$$

where λ is sampled from $U(0,1)$. The new point is therefore a random point on the segment between x_i and x_{nn}.



New points fill the space between nearby minority samples. This tends to make the minority-class region more general and less dependent on isolated observations. However, synthetic samples may also create ambiguous samples near class overlap, so the data must be inspected and validated carefully.

11 Lesson 11 — 12/11/2026

11.1 Clustering

`Clustering` is an unsupervised machine learning method: we have neither a ground truth nor labeled instances. It relies on one simple idea: identify groups of objects that are similar to each other and different from the others.

Why should we use clustering?

- `Explore structure`: reveal natural groups and summarize how the data are distributed. Examples: customer segments, biological taxonomy, document collections.
- `Preprocess data`: create compact representations or localize subsequent analysis. Examples: compression, vector quantization, local regression or classification.
- `Detect unusual cases`: objects that are far from any cluster can be candidates for anomalies or outliers.

Clustering can be the final analysis, or just one step inside a larger machine learning pipeline.

Of course one of the most important features in clustering is the `distance`: most clustering algorithms look for similarity using a distance, so we need to choose a distance that matches the data type and its semantics.

The main clustering strategies used in the literature are:

- `Partitioning`: iteratively improve a partition of the data. Examples: k-means, k-medoids.
- `Hierarchical`: build a nested decomposition by merging or splitting groups. Examples: AGNES, DIANA, BIRCH.
- `Density-based`: discover connected dense regions, so clusters can have arbitrary shapes. Examples: DBSCAN, OPTICS.

- `Model-based`: assume a model for each cluster and fit it to the data. Examples: EM, SOM.

Before clustering there is a question we need to ask ourselves: do the data already contain some natural structure? If we run a clustering algorithm without checking whether the dataset has a `clustering tendency`, we will obtain some clusters anyway, but are they reliable? Only if the dataset has a clustering tendency. We can use the `Hopkins statistic` to evaluate the clustering tendency of a dataset.

The intuition behind the Hopkins statistic is to compare two samples: points selected from the data versus points generated uniformly over the same space. If the data are clustered, uniform points tend to fall in empty regions.

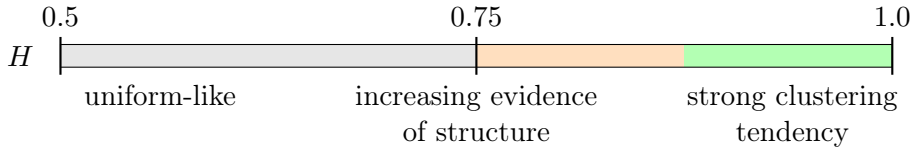
The computation follows three steps:

1. *Sample from D*: select $n \ll N$ data points p_i. For each p_i, compute x_i as the distance to its nearest neighbor in D.
2. *Sample uniformly*: generate n uniform random points q_i over the same data space. For each q_i, compute y_i as the distance to the nearest point in D.
3. *Combine distances*: compare the total random-to-data distance with the total data-to-data distance.

Definition 11.1 (Hopkins statistic)**.**

$$H = \frac{\sum_{i=1}^{n} y_i}{\sum_{i=1}^{n} x_i + \sum_{i=1}^{n} y_i}$$

The calculation is repeated for several random samples and we use the average H.



- $H \approx 0.5$: the data behave similarly to a uniform random distribution.
- H increases: random points are farther from the data than sampled data points are from one another.
- Rule of thumb: $H > 0.75$ indicates a clear clustering tendency.

In terms of hypothesis testing, H_0: D is uniformly distributed (no meaningful clusters), H_1: D is not uniformly distributed. The statistic is a quick check for clustering tendency, not a clustering algorithm.

11.2 Partitioning algorithms

The first thing we need to do is decide how many clusters we want to obtain, k. We then cut the data into k distinct groups and choose a representative for each group:

- `Centroid-based`: the representative is the mean of the points. Efficient and widely used for continuous features.
- `Medoid-based`: the representative is an observed object. More interpretable and works with arbitrary dissimilarities.

11.2.1 k-means

In the `k-means` algorithm we follow a simple procedure: assign points, update centroids, repeat.

First of all we need an initialization phase where we arbitrarily choose k objects from the dataset as initial centroids c_i. We then follow these three steps:

1. *Assign*: send each point to the nearest centroid c_i.
2. *Update*: replace each centroid c_i by the mean of its assigned points,

$$c_i = \frac{1}{|C_i|} \sum_{p \in C_i} p.$$

3. *Repeat*: stop when the assignments do not change or the improvement is tiny.

The objective is to minimize the total squared distance to the cluster centroids:

$$E = \sum_{i=1}^{k} \sum_{o \in C_i} \|o - c_i\|^2 .$$

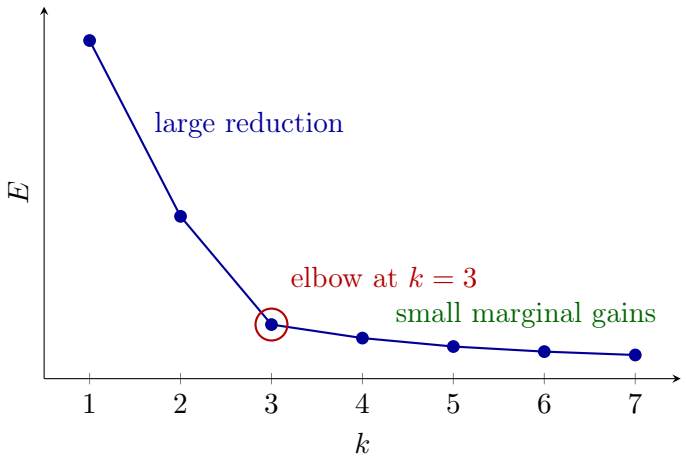
The mean is optimal only for the squared Euclidean distance in a continuous vector space. K-means is fast because each step is easy, but it is not guaranteed to find the global optimum.

K-means is still widely used because it is usually fast and scalable: the complexity is $O(tkn)$, where t is the number of iterations, so it is linear in the number of points. It is also easy to interpret and useful as a summarization tool.

The limitations of k-means and the current fixes are:

Limitation	Fix
Sensitive to initialization	Run several restarts; use k-means++ initialization.
Sensitive to scale	Standardize variables; choose features and weights carefully.
Sensitive to outliers	Trim outliers, use robust preprocessing, or switch to medoids.
Assumes convex clusters	For arbitrary shapes, use density-based or graph-based clustering.
Requires k	Use the elbow as a diagnostic; validate with silhouette, stability or domain knowledge.
Numerical features only	For categorical or mixed data, consider k-modes, k-prototypes or medoids with an appropriate dissimilarity.

Another problem in k-means is of course choosing k. One of the most popular methods is the heuristic known as the `elbow method`: we iteratively increase k starting from one, and for each k we compute the error E. If we plot the value of the error we will find an elbow in the curve, and that will be our k.



The key idea is that we choose the smallest k after which the error decreases only slowly. We don't want to simply minimize the error, because its lower bound is 0, reached when k is equal to the number of points: increasing k will always reduce the error, but it doesn't mean that the clustering is better. That's why we look for the elbow and not for the minimum.

11.2.2 k-medoids

One aspect of k-means we need to have clear is that it is sensitive to outliers: this derives from the fact that we compute the average to choose the centroid. To prevent this we can use the `medoid` instead: the medoid is the most central object of the cluster, i.e. the real object that minimizes the total dissimilarity to the other objects of the cluster. A single extreme point cannot pull a medoid away like it can pull a mean, so the medoid is much more robust to outliers.

Remark. The medoid is not the median. The median is computed attribute by attribute and, with more than one attribute, it is in general not an object of the dataset; the medoid is always an observed object. The two coincide only in special cases (e.g. one-dimensional data with an odd number of points).

For the `k-medoids` method we can follow the same steps as k-means; the difference is that we don't compute a synthetic representative: we can only move the medoid from a real instance to another real one. This leads to a slower algorithm, because in the worst case we need to try all the points as medoids.

In the literature we have a lot of k-medoids algorithms. They all share the same objective, what changes is the search strategy and its scalability:

- `PAM / FasterPAM`: exact local-search idea, try the swaps that reduce the total dissimilarity. Modern variants are much faster but still need many distances. Suited for small or medium n.
- `CLARA` (sampling): run k-medoids on samples, then evaluate the medoids on the full data. Useful for large n when a representative sample exists.
- `CLARANS` (randomized search): explore a random subset of swaps instead of all the neighbors. Suited for a large search space.
- `BanditPAM` (approximations): use adaptive sampling or approximate distance evaluation to reduce the object-pair checks. Suited for very large n.

`PAM` (Partitioning Around Medoids) works in two phases. The *BUILD* phase chooses a useful starting set with a greedy initialization:

1. *Choose the first medoid*: test every object as the sole representative and keep the one with the smallest sum of distances to all objects,

$$m_1 = \arg\min_h \sum_i d(x_i, h).$$

2. *Add one medoid at a time*: for each non-medoid h, measure how much it improves the current nearest-medoid distances,

$$gain(h) = \sum_i \max\{0,\ D_i - d(x_i, h)\},$$

where D_i is the distance from x_i to its current nearest medoid, and add the candidate with the largest gain.

BUILD is a strong initialization, not a guarantee of the global optimum: PAM still needs the SWAP phase.

The *SWAP* phase improves one representative at a time. A neighbor solution replaces one current medoid with one non-medoid:

1. *Assign*: store each object's nearest current medoid and the total cost T.
2. *Evaluate*: for every medoid m and non-medoid h, compute the change of cost $\Delta(m \to h)$.
3. *Update*: perform the swap with the most negative Δ, then reassign the objects.
4. *Stop*: if every candidate swap has $\Delta \geq 0$, the solution is a local optimum.

PAM examines the full one-swap neighborhood, but it can still stop at a local, not global, minimum.

The other algorithms reduce the cost of this search in different ways:

- `CLARA` scales PAM by sampling the objects: it runs BUILD and SWAP only within a small sample, scores the resulting medoids on the full dataset, and repeats for several samples keeping the medoids with the lowest full-data cost. The main risk is that a good medoid, or a small rare cluster, cannot be discovered if it is absent from every sample.
- `CLARANS` samples the search neighborhood, not the data: it proposes a random swap, moves immediately when the cost improves, and after a chosen number of failed random swaps treats the current solution as a local minimum; then it restarts from another random medoid set and keeps the best local minimum found.
- `BanditPAM` samples the distance evaluations adaptively: candidates are treated like competing arms, their average cost is estimated from a growing subset of distances, and a candidate is discarded as soon as its optimistic bound is worse than another candidate's pessimistic bound.

To decide between k-means and k-medoids we choose the representative that matches the data and the goal:

- *Use k-means when* data are numerical, scaled, large, and clusters are expected to be compact around average profiles.
- *Use k-medoids when* representatives must be actual objects, outliers matter, or the distance is not Euclidean.
- *Use something else when* clusters are non-convex, density varies strongly, or the number of clusters is not naturally fixed.

	k-means	k-medoids
Speed on vectors	high	medium
Robustness to outliers	low	high
Interpretability of the representative	medium	high
Arbitrary distances	low	high